SIGNAL EXPLORER

User's Guide

CAN / CAN FD, LIN and Ethernet desktop workbench

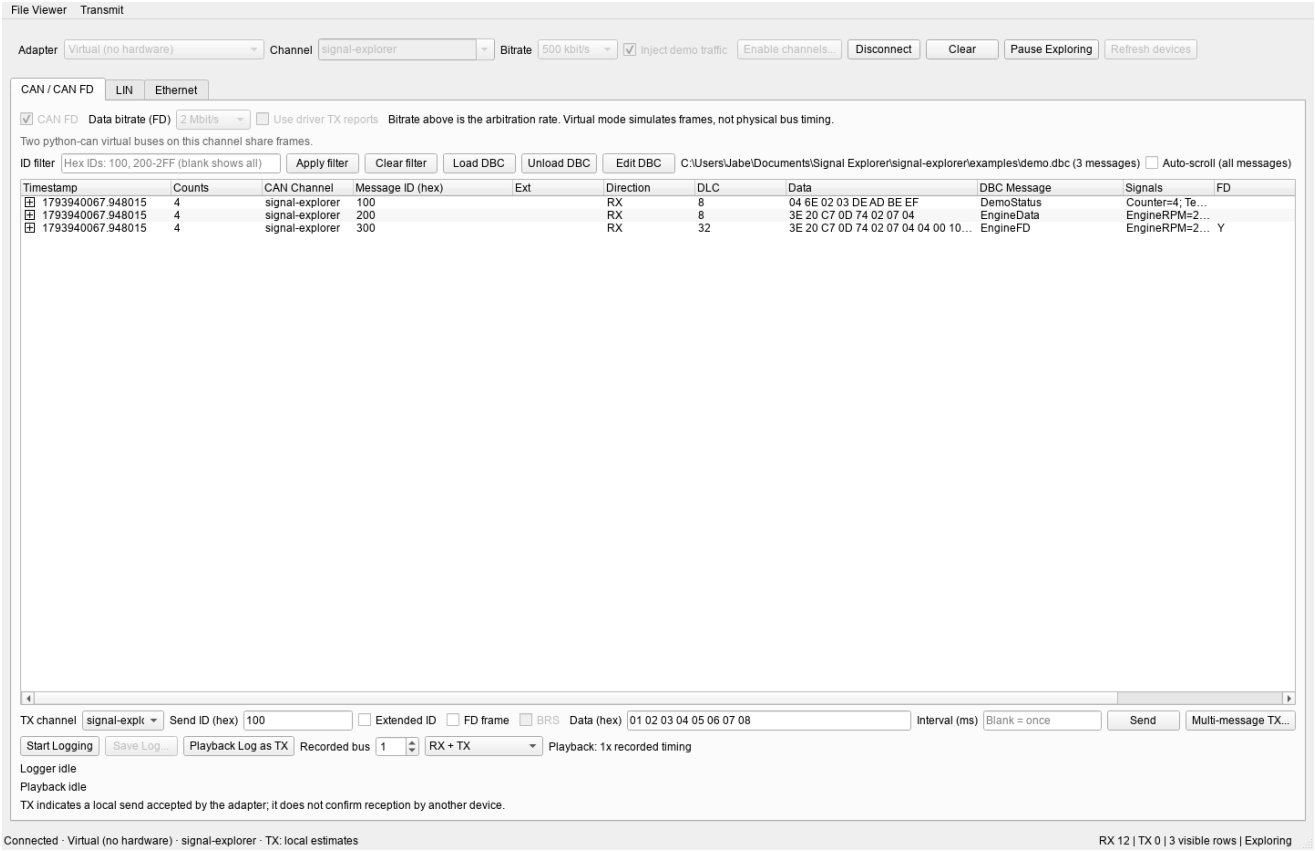


Figure 1: Signal Explorer running virtual CAN FD traffic with the demo DBC.

Windows driver setup and NI USB-8473 Classic CAN
Document revision 1.9 - November 13, 2026

Note - This guide describes the current source build. Hardware driver integration is not a physical-bus qualification. NI-XNET USB-8502, native LIN and ValueCAN Ethernet await powered bench verification. Passive PC Ethernet capture has been checked. Ethernet TX and NI-CAN USB-8473 frame I/O await physical testing. A Windows installer beta is available; vendor drivers remain separate.

Read this guide alongside Install guide.pdf. Install guide covers deployment; this manual covers operation, hardware selection, verification and recovery.

Contents and document scope

Section	Pages	Use it for
November 5 release changes	4	Batched CAN display, script buffering and timing.
1. System prerequisites & hardware profile	6-8	PC requirements, rates, FD and isolation.
2. Mandatory driver/software setup	9	Preparation before USB or bus connection; Npcap.
3. Connection & verification	10-15	CAN, NI USB-8502/8473, LIN and Ethernet checks.
4. Graphical interface reference map	16-27	Columns and controls, including the DBC editor and Signal Graph.
5. Practical workflows	28-34	DBC editing/decoding, LDF, sending and MF4 replay.
6. Troubleshooting & fault codes	35-39	CAN/LIN/Ethernet faults, driver errors and recovery.
7. Appendix: adapter pinouts	40-47	CANable, Kvaser, PEAK, Vector, Intrepid, NI and Ethernet.
7. Appendix: command line and sources	48-50	Launch commands, environment settings and source register.

Table 1: Navigation for this edition.

How to read the support statements

Implemented means there is an application code path. **Mock tested** means API behavior was checked without electrical traffic. **Physically observed** means the specific observation stated was made; it does not imply all rates, adapters or features are qualified.

This release has 219 passing automated tests, including adapter preferences and DBC editing. Existing CANable reception is distinct from native driver mock coverage. ValueCAN4-2EL VE0245 and five PC Ethernet interfaces were discovered on the development PC. Passive capture on its active Ethernet port worked; no application-generated physical Ethernet TX or powered ValueCAN Ethernet/LIN verification was performed.

Manual conventions

Control names are bold. CAN and LIN IDs and data bytes are hexadecimal; rates and intervals are decimal unless marked otherwise. Connector tables refer to pin numbers molded into the connector, not wire colors. NC means not connected; a blank vendor assignment is identified explicitly.

This manual uses the supplied Kvaser Leaf Light R v2 guide as a layout reference: numbered sections, restrained typography, figures, tables, notes and running page headers. Kvaser logos and product photographs are not reused. The Signal Explorer screenshots contain simulated traffic.

October 29, 2026: release changes



Application version remains 0.1.0. Document revision 1.2 covers the current source build; it does not introduce an installer.

Change	Current behavior
File Viewer menu	Replaces Scripts. New/open Python scripts, the CAN simulation example and Open Logfile are available.
Python editor and runner	Separate window, syntax colors, line numbers, Find/Replace, Save/Save As, Run/Pause/Resume/Stop, line delay, Follow execution and Script Output.
Continuous demo test	Standard 321-hex requests contain eight bytes; the virtual peer replies on 221 hex with the same data. The script verifies, increments every byte and repeats until stopped.
Logfile viewer	Read-only MF4 event rows. Replay RX, TX or both through the connected protocol, with timed highlighting, Pause/Resume/Stop and Replay Output.
Replay timing	1x recorded gaps. Log line delay samples highlighting only; Python line delay slows actual source execution. Playback ownership is shared with the main protocol tab.
Icon and splash	Black SE icon with red S and white E. Startup splash adds white SIGNAL and EXPLORER; it closes when the main window opens. The title-bar icon stays the SE mark.
Intrepid timestamps	CAN, LIN and Ethernet RX hardware intervals are anchored to host time on each connection, aligning them with local TX timestamps. Invalid replay clock jumps fail preflight.

Table 2: Changes implemented October 29, 2026.

The separate **Scripting User's Guide.pdf** introduces Python and provides commented, downloadable source examples for CAN/MCU tests.

November 5, 2026: busy-bus improvements

Area	Current behavior
CAN display batching	A bounded 20000-observation queue drains up to 2000 every 40 ms. Model row ranges update once per batch, counters about 10 times/s and scrolling once per batch. Display drops reports overwritten queued observations separately from logging.
DBC and signal tree	Indexed ID/format definitions and lazy decode caches shared by summary/child cells. Constant-time history row and signal-parent lookup. Latest values, channel counts and history switching are preserved.
Script receive history	The separate 2000-frame script ring retains nonmatching traffic. Waiting for A no longer consumes B. Full buffers drop oldest entries during sleep/pause and warn instead of automatically stopping the script.
Script buffer controls	se.can.stats() reports buffered/received/consumed/dropped/filtered/flushed and capacity. flush() clears history without resetting loss. Check dropped before accepting test evidence. Counters reset per Run.
Script capture filters	set_filters accepts ORed can_id/can_mask dictionaries, optionally extended/direction/channel. These affect only this script; None/[] restores all data frames. Changes prune buffered nonmatches; invalid edits preserve the old configuration.
TX and timing	send returns host acceptance time. Optional driver TX reports replace synthetic TX; receive a TX event for report time. Monotonic host estimates resist PC clock changes. See section 6.4 for clock limits and recording diagnostics.

Table 3: No new runtime dependencies; CAN/CAN FD scripting remains the only Python bus transport.

Verification

230 automated tests pass, including high-load sleep/pause buffering, out-of-order A/B receives, loss counters, filters/flush, TX timestamp identity and notifications without polling. A 4000-frame virtual-bus burst records every frame while the trace retains its configured history. Local 20000-frame model ingestion improved from 4.39 s to 0.14 s; this is a synthetic model benchmark, not physical throughput qualification.

The Scripting User's Guide revision 1.3 and docs/scripting.md show the new API. Script Output includes a loss warning and final drop total. A script can still fail its own checks, timeout, fail on disconnect or overflow its output; buffer loss is now an explicit test-policy decision.

November 2, 2026: release changes

Change	Current behavior
Selective CAN channels	Enable channels... lists physical devices and individual CAN ports. Open only the checked ports, sharing bitrate and Classic/FD timing. Choose one TX channel.
Saved selections	The last device and checked channels are remembered per backend/device for this Windows user. New devices start with their first available channel. Restoration never connects or sends.
Trace defaults	CAN Channel replaces Bus Number. Same-ID traffic stays separate by channel. CAN Auto-scroll starts unchecked: latest-value rows are the default. LIN/Ethernet defaults are unchanged.
Multi-message sender	Separate Send Messages window retains the main single-message sender. Independent raw/DBC rows, once/cyclic sending, per-row channels, FD/BRS and signal waveforms.
Live editing	Edit running rows, bytes, signals, cycle times and channels. Valid changes apply to subsequent sends; invalid changes leave the prior configuration running. An in-flight frame may use old values.
Portable TX lists	Open TX List and Save TX List buttons plus File menu operations save JSON configuration, embedded DBC definitions and waveforms. Loaded lists never start themselves. Clone remains; cut/copy/paste are removed from this window.
Recording and scripts	MF4 preserves separate CAN BusChannel values and an embedded channel map. Python send/recv accept an enabled channel label or native ID. Replay selects one source bus and one TX destination.

Table 4: November 2 source build; application version remains 0.1.0.

Validation and hardware scope

230 automated tests pass. One physical VN1630A test at 500 kbit/s Classic CAN sent standard ID 5A1 on CAN1 and received identical bytes on CAN2. Only those two ports were opened and the session was released afterward. Physical CAN FD and the new transmit-list workflows remain unqualified; FD routing and multi-message behavior use virtual/mock tests.

The Intrepid neoVI FIRE 3 profile exposes 16 CAN networks and lists 8 LIN networks in inventory. This is mock-tested CAN discovery/routing, not a new FIRE 3 LIN transport. Existing native LIN/Ethernet qualification limits still apply.

1 System prerequisites & hardware profile

1.1 PC and software prerequisites

• Source setup uses Windows 10/11 x64 and **64-bit Python 3.12**. The installer beta bundles Python and targets Windows 10 1809+ or Windows 11 x64. The package declares Python 3.11+; that is not a cross-version test guarantee.

• Keep the project in a writable folder and create its own .venv. Allow disk space for Python packages, temporary captures and saved MF4 files; no sustained-load RAM/storage minimum has been qualified.

• The main packages are PySide6-Essentials, python-can[mf4], asammdf, pyserial, cantools, uptime and ldfparser. Intrepid adds python-ics and filelock.

• Obtain the correct bus description, connector/cable part number, baud rates and a known-good second node before physical testing. Hardware detection alone does not prove a healthy bus.

1.2 Rates actually selectable in this application

Setting	Current choices and effect
CAN arbitration bitrate	125, 250, 500 or 1000 kbit/s; default 500 kbit/s. The GUI does not offer custom sample-point or arbitrary bitrate editing.
CAN FD data bitrate	0.5, 1, 2, 4, 5 or 8 Mbit/s; default 2 Mbit/s. Must be at least the arbitration rate; the adapter/driver may reject a choice.
LIN baud rate	2400, 4800, 9600, 10400, 19200 or 20000 bit/s. The demo uses 19200. Loading an LDF requires an available rate.
Repeat interval	CAN/Ethernet: blank sends once; 1 to 2147483647 whole milliseconds repeats. LIN uses schedule slot delays.
Ethernet link / frame size	ValueCAN4-2EL: 10/100 PHY. PC rate is controlled by Windows/NIC settings. App MTU: 1500 bytes; Ethernet II with one VLAN tag. No speed/duplex or CAN FD setting applies to Ethernet.

Table 5: GUI settings, not vendor hardware maxima.

1.3 CAN FD and electrical prerequisites

Enable **CAN FD** before connecting, then choose **FD frame** for each outgoing FD message. **BRS** switches to the data bitrate. Valid FD byte counts are 0-8, 12, 16, 20, 24, 32, 48 and 64. Classic CAN is limited to 8 bytes. Standard IDs use 11 bits; Extended ID uses 29.

SLCAN and gs_usb remain Classic CAN paths in this build, even if a particular board/firmware advertises FD. ValueCAN3 and Leaf Light R v2 are Classic-only. A classic-only node may disturb FD traffic; confirm the entire network configuration before enabling FD.

Note - CAN requires correct end termination and a functioning peer. LIN requires a suitable bus supply and one master. Isolation is a hardware property, not an app setting. A test voltage is not a permissible continuous working voltage.

1.4 Supported hardware and isolation

Adapter / family	Application support	Hardware profile / isolation
CANable original / 2.0 / Pro	Classic CAN via SLCAN; candleLight via gs_usb. No LIN.	Board/clone dependent. Pro is advertised with 2.5 kV isolation; do not transfer that rating to original/2.0/clones. Use actual silkscreen and board documentation. [H1]
Vector VN1610	CAN and FD via XL; no LIN on its CAN transceivers.	Two capacitively decoupled 1051cap CAN channels on one D-Sub. Vendor: CAN up to 1 Mbit/s, FD 5 Mbit/s (8 with suitable wiring/transceivers). No isolation voltage asserted here. [H5]
Vector VN1630 / VN1630A / VN1640 / VN1640A	CAN/FD where fitted hardware supports it; native XL LIN on active LIN-capable piggies.	Transceiver-dependent. 7269mag example: 50 V insulation rating; other piggies differ. Match exact revision; the app label CANXL does not add CAN XL. [H5,H6]
Intrepid ValueCAN3	Classic CAN via neoVI; no FD or LIN.	Two CAN channels, USB isolation; exact isolation test voltage not stated in the cited profile. [H4]
Intrepid ValueCAN4-1/-2/-4	CAN/FD via neoVI. No LIN on these variants.	ValueCAN4 family has isolated USB. Do not assume per-channel isolation. Hardware FD up to 8 Mbit/s subject to model/timing. [H3]
Intrepid ValueCAN4-2EL	CAN/FD; experimental native LIN 1 and Ethernet 1.	USB supports CAN; external 12 V for LIN/Ethernet. 10/100 PHY. USB isolation; no voltage rating claimed. Cable variant changes pins. [H2,H3]
Kvaser U100-C	CAN/FD via CANlib; no Kvaser LIN backend.	CAN 10-1000 kbit/s; FD up to 8 Mbit/s. Reinforced isolation tested at 5000 VAC rms for 60 s. No internal CAN termination. [H7]
Kvaser Leaf Light R v2	Classic CAN via CANlib; FD must stay off. Not physically tested here.	50-1000 kbit/s, USB powered, galvanically isolated; supplied manual gives no numerical isolation rating. No internal termination. [H8]
PEAK PCAN-USB Pro FD	CAN/FD via PCAN-Basic; experimental LIN via PLIN.	Hardware: CAN 25-1000 kbit/s, FD data up to 12 Mbit/s; app stops at 8. CAN ports isolated up to 500 V from USB/LIN. LIN pair: 500 V from USB/CAN, not each other. Each LIN port needs 8-18 V. [H9]
Virtual CAN / LIN / Ethernet	Hardware-free functional examples.	No electrical connector, isolation, bus-off behavior or physical timing simulation. PC Ethernet capture uses the NIC; its isolation/rate follow its own hardware manual.

Table 6: Named hardware profiles. Vendor limits can exceed the GUI choices in section 1.2.

Native LIN for Vector, Intrepid and PEAK has mock coverage; powered timing, response removal, error reporting and interoperability still need bench qualification. Kvaser U100-C and PEAK hardware were not available for physical tests. Other devices accepted by a generic vendor backend require their own manual and validation.

1.5 NI USB-8502 hardware profile

The NI (USB-8502 / NI-XNET) CAN backend supports the one-port and two-port USB-8502 high-speed CAN / CAN FD variants. It uses NI-XNET frame streams through python-can. The USB-8473 family uses the separate Classic CAN NI-CAN backend described in section 3.8.

Item	Profile / application behavior
Device and power	USB-8502: one or two 9-pin male D-Sub CAN ports. USB supplies the device; external CAN transceiver power is not needed. The optional three-pin connector on the two-port unit is for synchronization, not a LIN channel. [N1,N2]
Selectable rates	Arbitration: 125, 250, 500 or 1000 kbit/s. FD data: 0.5, 1, 2, 4, 5 or 8 Mbit/s, subject to driver/hardware acceptance. Start at 500 kbit/s / 2 Mbit/s.
Vendor physical limits	NI lists 1 Mbit/s arbitration and a TJA1043T transceiver certified for FD data up to 5 Mbit/s; 8 Mbit/s is experimental for that revision. Newer hardware revisions can differ. An app menu choice does not qualify a bus. [N3]
Termination	Switchable nominal 120-ohm internal resistor. NI internal termination defaults off. Enable before connecting only when this port needs an end resistor. Discovery disables the control for a port without this capability. [N3]
Isolation	NI specifies isolation between ports on the two-port model and between CAN and USB. Continuous port-to-earth rating: 60 VDC, Category I. COM is isolated from USB ground; this is not UL safety isolation. [N3]
Integrated features	RX and local TX trace, once/repeated TX, filters, DBC decoding, Python CAN scripts, MF4 logging and timed replay use the shared CAN service. No NI LIN, Ethernet or synchronization configuration is added.

Table 7: USB-8502 integration and electrical profile.

Note - NI-XNET interfaces with a high-speed/FD transceiver are discoverable. The USB-8502 is the target device family; other NI models require their own capability checks and bench qualification. Mock coverage does not establish physical traffic, rate or timing performance.

Source: [N1] NI USB-8502 product page; [N2] Quick Start 376701A; [N3] NI USB-8502 Specifications (updated June 23, 2025). Links accompany the NI pinout appendix.

2 Mandatory driver/software setup

2.1 Before plugging the adapter or bus in

1. Identify the exact adapter, firmware personality (SLCAN vs candleLight), piggy type and cable. Save the matching pinout and bus parameters.

2. Install the manufacturer's Windows driver and API runtime per its prompts. Keep vehicle/bus wiring disconnected. The USB-8473 family uses a 32-bit NI-CAN API.

3. Run Windows Setup or follow the Install Guide for a source installation. Open **Help > About Signal Explorer** to find the version and driver download links after installation.

4. After installation, connect USB and verify enumeration in Device Manager and the vendor utility. Connect the physical bus only after checking the correct cable map and supply.

Backend	Required software before connection
CANable SLCAN	Correct serial firmware and Windows USB serial driver if needed. The COM number is machine-specific. App serial speed is 115200; this is not the CAN bitrate.
CANable / candleLight	Compatible gs_usb firmware plus python-can[gs-usb], USB/WinUSB/libusb setup appropriate to the board. Changing the USB driver can break SLCAN access.
Vector CAN / LIN	Vector hardware drivers plus XL Driver Library. If not found, use Vector DLL in the CAN tab to choose vxlapi64.dll and restart.
Intrepid CAN / LIN / Ethernet	Intrepid driver, 64-bit icsneo40.dll and [intrepid] optional packages. Ethernet: current python-ics with ExtraDataPtr fixes; 1!26.3.9+ recommended, 1!26.3.17 checked.
PC Ethernet (Windows)	Install Npcap from npcap.com. The app loads installed System32/Npcap/wpcap.dll. No extra Python capture package. Admin-only Npcap installs may require elevation. [E1]
Kvaser CAN	Kvaser Windows drivers with CANlib runtime. Both U100-C and Leaf Light R v2 use CANlib; a CANlib virtual channel is not physical hardware.
PEAK CAN	PEAK driver and 64-bit PCAN-Basic API (PCANBasic.dll).
PEAK LIN	PEAK LIN driver, PLIN Manager service and PLIN-API (PLinApi.dll). PCAN-Basic alone cannot supply LIN.
NI USB-8502 CAN/FD	NI-XNET Windows driver and NI MAX; source users install the [nixnet] Python extra. See section 3.6.
NI USB-8473/8473s	NI-CAN 18.5 with NI MAX. Windows Setup bundles the x86 helper. See section 3.8.

Table 8: Optional vendor components are independent.

Note - Signal Explorer does not update adapter firmware. Record the installed driver/firmware versions before changing them and use the vendor's update utility if required.

3 Connection & verification: CAN

3.1 Hardware-free application check

1. Launch the app, choose **Virtual (no hardware)**, leave Inject demo traffic checked, and Connect.

2. Expect IDs 100 and 200 to change. Load examples/demo.dbc to decode DemoStatus and EngineData. Disconnect, enable CAN FD and reconnect to add 32-byte ID 300.

3. Send ID 200 with DE AD BE EF and a blank interval. Expect one local TX row. This validates the UI/send path, not external electrical reception.

Note - python-can Virtual buses exchange within one Python process. Use the built-in demo peer to simulate responses. Listen-only is available on selected hardware backends.

3.2 Verify hardware in the vendor console

Adapter	Independent check before using Signal Explorer
Vector	Open Vector Hardware Config / Vector Hardware Manager. Confirm serial, bus transceiver and active channel. Use the vendor loop test if installed, following its required two-channel cable/termination setup.
Intrepid	Use neoVI Explorer / ICS Device Manager or Vehicle Spy to confirm serial, CAN/LIN enables and bitrates. The EL needs external supply for LIN. Stop scripts and release the device before app connection.
Kvaser	Use Kvaser Device Guide to confirm model/channel and a nonzero firmware version. CanKing provides an independent traffic check. Leaf R v2 green PWR should be steady; yellow CAN indicates traffic. [H8]
PEAK	Use PEAK-Settings to check CAN Hardware and LIN Hardware, then PCAN-View or PLIN-View Pro as appropriate. Release the LIN channel before Signal Explorer. [H9]
CANable	Confirm the expected COM/USB device and firmware. Close serial terminals or other bus tools before SLCAN connection. USB enumeration does not establish CAN traffic.

Table 9: Device-console checks; these utilities are not embedded in Signal Explorer.

3.3 Physical two-node CAN verification

Use a known-good second adapter or ECU, CAN-H to CAN-H, CAN-L to CAN-L and the correct bus-side reference. Put 120-ohm termination at both ends; about 60 ohms between H and L with power off is a useful check when no additional paths affect the reading. Match arbitration rate and FD/BRS/data rate on both nodes.

Refresh devices, select the exact channel and Connect. Send a known frame from the peer and confirm RX; send from Signal Explorer and confirm it at the peer. A lone normal CAN transmitter needs another active node to acknowledge it. TX rows alone are not proof of ACK or successful peer receipt. [F1]

3.4 Connection & verification: LIN

Virtual LIN first

1. Choose the LIN tab. Leave Virtual LIN (simulated), Monitor, 19200 and Simulated peer selected. Connect LIN; expect header/response events for IDs 10 and 20.

2. Disconnect, select Master and reconnect. Send Header 20 or Start Schedule; Stop Schedule independently of Pause Exploring.

3. Disconnect, choose Slave and reconnect with Simulated peer. EngineStatus responses transmit only when the peer master requests them. Change response data and Apply Responses.

Native adapter connection

Choice	Expected selection / setup
Intrepid ValueCAN4-2EL	Refresh and choose the detected serial, e.g. VE0245, LIN 1. Model type is checked; ValueCAN4-2 and ValueCAN3 are excluded. Temporary baud/resistor changes use no EEPROM writes.
Vector VN16xx / XL LIN	Refresh and choose an active LIN-capable channel with the correct piggy. Channel index/mask are taken from discovery. CAN-only VN1610 channels are not listed. Release other tools' initialization access.
PEAK PLIN / PCAN-USB Pro FD	Refresh and choose the detected PLIN channel, not a guessed PCAN CAN channel name. Close other PLIN clients on that LIN channel.

Table 10: Native LIN choices. USB detection is not a powered transmission test.

Powered bench acceptance sequence

1. Use the exact appendix cable map. Provide the vendor-specified bus supply, common bus reference and the proper master pull-up/diode arrangement. LIN does not use CAN's two 120-ohm terminators.

2. Use CANalyzer/VN1640 as master and Signal Explorer/ValueCAN4-2EL as Slave. Load matching LDF, rate, length and checksum settings. Confirm returned bytes on the external tool.

3. Reverse roles when the external tool's license/configuration supports responder simulation. Confirm Master Header, local responses, schedules, unchecking/removing responders and stopping without unintended TX.

4. Check no-response, checksum errors, pause vs logging, stop/disconnect cleanup and MF4 import. Intrepid dedicated Wake is disabled; use the external master to wake that bus.

> **Note** - Native Vector CAN/LIN and Intrepid CAN/LIN/Ethernet sessions on the same device are mutually exclusive within this app until shared native sessions are implemented. Separate devices can stay connected in separate tabs. Powered native LIN behavior is not qualified.

3.5 Ethernet connection & verification

Choose a transport; the simulator stays available

Choice	Prerequisites / profile
Simulated Ethernet	No hardware or capture driver. In-process UDP/ARP/VLAN/raw test traffic and an optional simulated peer.
PC Ethernet | port name | MAC | up/down	Windows and Npcap. Lists wired and virtual Ethernet interfaces, including unplugged ports; Wi-Fi and loopback are excluded. NIC rate, isolation and connector follow its own manual, not a generic app rating.
Intrepid ValueCAN4-2EL | serial | Ethernet 1	Intrepid driver/API plus python-ics; external 12 V and the matching Ethernet breakout. 10/100 PHY; preserve/configure existing speed/duplex in neoVI Explorer. CAN-only ValueCAN variants are excluded. [H2]

Table 11: Ethernet transport selection. A capture session being open is not link verification.

1. Open **Ethernet**. The first display scans adapters; **Refresh adapters** rescans while disconnected. Simulation is the default. Discovery never opens or transmits on an interface.

2. For a software check, leave Inject demo traffic and Simulated peer replies checked and Connect Ethernet. Expect UDP telemetry, ARP, VLAN UDP and raw 88B5 frames. Send a UDP test to obtain a synthetic ACK response.

3. For PC capture, select the intended port/MAC and inspect link status. Enable Promiscuous capture only when needed, then Connect Ethernet. Capture starts automatically; verify incoming traffic from a known peer. No app-generated TX is needed for this check.

4. For ValueCAN, disconnect CAN/LIN or other tools using that unit. Provide 12 V and verify PHY/link in the vendor console; release the console and Connect Ethernet. Device detection over USB alone is insufficient.

5. For a physical TX bench test, use a peer capture tool, choose a raw test frame, enter the peer destination MAC and suitable source/data, then Send once. Confirm the complete bytes at the peer before testing repeat or replay. This acceptance test remains pending.

What capture can and cannot establish

PC direction is inferred from the selected NIC source MAC and includes other PC applications' traffic. Promiscuous mode sees only frames delivered to that port; use a switch mirror or TAP for other switch ports. ValueCAN uses native RX records and locally accepted TX. TX is not proof of peer reception. [E1,E2]

> **Note** - Native hardware never receives the simulator's automatic demo traffic or ACK replies. Send, repeated send and playback transmit on the selected physical port. No PC IP-address changes, DoIP activation-line control or automatic ARP/IP stack are provided.

3.6 NI-XNET connection & verification

Install before connecting USB

Install NI-XNET for Windows with support for your Windows version and USB-8502. Include NI Measurement & Automation Explorer (MAX), follow installer prompts and restart if requested. The NI Python binding is separate from the native driver; install it into the application environment:

```
.\.venv\Scripts\python.exe -m pip install -e ".[nixnet]"
# Both optional vendor bindings:
.\.venv\Scripts\python.exe -m pip install -e ".[intrepid,nixnet]"
```

Verify in NI MAX, then connect in Signal Explorer

1. Connect USB. In NI MAX, locate the USB-8502 and note the exact CAN interface names and physical ports. Names such as CAN1 and CAN2 are assigned by NI; do not assume Port 1 is always CAN1.

2. Use NI MAX Bus Monitor with a known-good peer and matching rates to confirm RX, as described in the NI Quick Start. Close Bus Monitor and any other session using that port before opening Signal Explorer.

3. Select **NI (USB-8502 / NI-XNET)** in the CAN adapter dropdown and **Refresh devices**. Choose model/serial/interface/port. A manual channel entry must match NI MAX exactly.

4. Start with Classic CAN and the peer bitrate. Set **NI internal termination (120 ohm)** only if this port needs an end resistor. Connect; monitoring starts. A silent bus produces no demo frames on hardware.

5. Confirm a known RX ID and bytes. Send a standard ID with eight known bytes, then confirm it at the peer. Test a 100-ms repeat and Stop. Local TX means the driver accepted a frame; it does not prove on-wire ACK or peer reception.

6. For FD, Disconnect, enable CAN FD, match arbitration/data rates and reconnect. Select FD frame, BRS and a 12-byte payload; verify it at an FD-capable peer. Save RX/TX to MF4, reopen it and check direction, bytes and timing before physical replay.

Diagnostics and acceptance status

Python API unavailable: install .[nixnet] with the same .venv interpreter. **Scan failed / DLL missing:** repair the 64-bit NI-XNET runtime. **No channels:** verify USB and NI MAX, then Refresh. **Connect failed:** check the interface name, rates, termination and competing sessions.

For bus-off or repeated errors, stop TX, check wiring, two bus-end resistors, reference ground, peer ACK and both FD rates, then Disconnect/reconnect. There is no NI-specific controller-reset button. Automated NI discovery/native conversion and MF4 checks use mocks; physical USB-8502 acceptance is pending.

3.7 Select and remember CAN channels

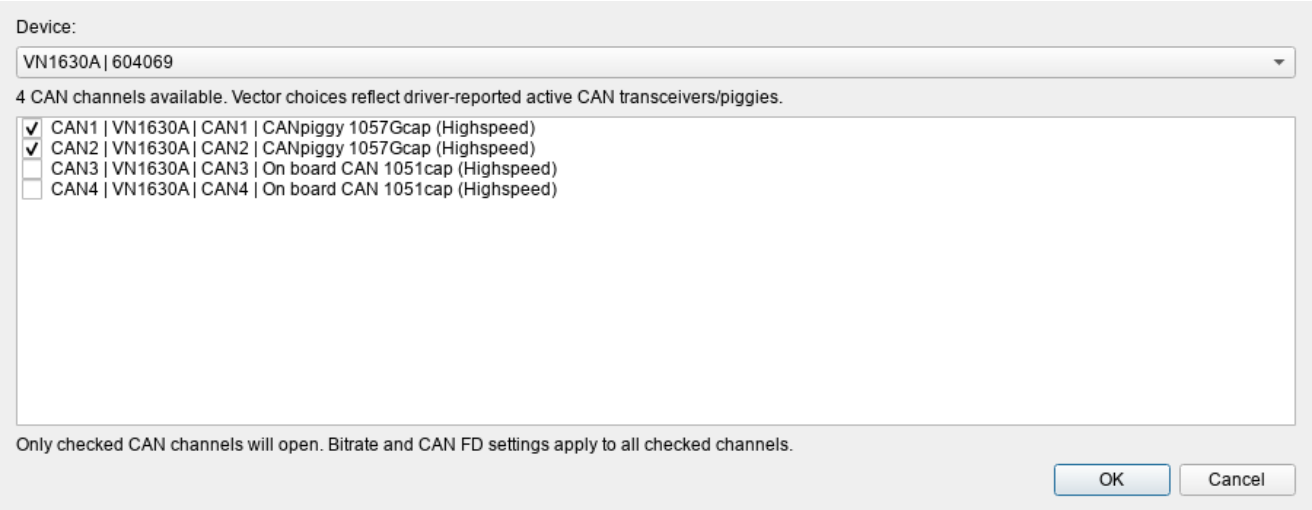


Figure: illustrative four-port Vector selection; only CAN1 and CAN2 are enabled.

1. While disconnected, choose the backend and Refresh devices. Click **Enable channels...**, select the physical model/serial and check only the required CAN ports. Confirm with OK.

2. Set a common arbitration bitrate, Classic/FD mode and FD data bitrate. All enabled ports share these settings. Known non-FD channels are rejected before an FD session opens.

3. Connect, then choose one enabled **TX channel**. Send targets that port only. The trace labels CAN1, CAN2, etc.; other native identifiers are shown when no physical label is available.

4. For a correctly wired/terminated CAN1-to-CAN2 loop, send on CAN1 and confirm a CAN2 RX with identical ID/data. A local CAN1 TX row alone is insufficient.

Restoration and missing hardware

The last physical device and checked subset are stored per backend/device in local Qt settings. They survive Refresh, reconnect and relaunch, including changed discovery order. A new device starts with its first channel enabled; the first application run still selects Virtual. No automatic connection or transmission occurs.

Missing saved ports are excluded while their preference is retained. If every saved port is unavailable, select available ports explicitly before Connect. Cancel leaves settings unchanged. Choosing the ordinary Channel dropdown deliberately replaces the saved subset with that single channel.

Device inventory boundaries

FIRE 3 is a CAN inventory/routing profile: 16 CAN networks plus 8 LIN inventory entries. No FIRE 3 LIN transport or electrical/pinout qualification is added. Use its exact vendor manual/harness; ValueCAN pinouts do not apply.

Vector inventory uses active CAN capabilities, physical port numbers and transceiver names. LIN-only piggies and I/O are excluded; Vector virtual ports are grouped separately. FIRE 3 CAN1-CAN16 use their true network IDs, including values above 255. Its LIN1-LIN8 inventory does not enable LIN sessions here. Unknown Intrepid models retain conservative/manual selection.

3.8 NI-CAN connection & verification

The one-port **NI USB-8473** and **USB-8473s** use high-speed Classic CAN. The standard Windows installer bundles the 32-bit helper. Install NI-CAN 18.5 and NI MAX separately. The Install Guide covers the additional x86 Python setup when running from source.

1. Connect the USB interface as directed by NI. In NI MAX, locate the device under Devices and Interfaces and note the CAN interface name, such as CAN0. Close its test panel before using the port in Signal Explorer.

2. Use **Detected devices** in the CAN tab, or select **NI (USB-8473 / NI-CAN)** and enter the same interface name manually. Select the bench bus bitrate and click Connect.

3. On a correctly wired and terminated bench bus, send a known frame from a second CAN node and compare its ID and bytes with the RX trace. Send a standard eight-byte frame in Signal Explorer and confirm reception at the other node. Disconnect and check that the port can be reopened.

Capabilities and troubleshooting

This backend provides active Classic CAN receive/transmit. CAN FD, Listen-only, driver TX reports and USB-8473s synchronization are outside its scope. A local TX row records adapter acceptance. Use the peer to verify on-wire reception.

If discovery fails, check USB, NI MAX enumeration and NI-CAN installation. Refresh the device list and check the interface name. If there are no frames, verify bus activity, bitrate, wiring and termination. NI lists Windows 10 for NI-CAN 18.5. NI MAX detected the device on Windows 11 in a user check. Signal Explorer RX/TX still needs bench verification.

NI-CAN driver download NI-CAN 18.5 requirements

4 Graphical interface reference map



Figure 2: CAN/CAN FD view. Demo traffic and decoded values are synthetic.

Region	What it contains
A. Above protocol tabs	Adapter, Channel, Enable channels..., Bitrate, Inject demo traffic, Connect/Disconnect, Clear, Start/Pause Exploring, Refresh devices and the conditional Vector DLL control.
B. Below CAN tab	CAN FD/data bitrate, conditional NI internal termination, driver notes, filter field, DBC controls and Auto-scroll.
C. Central trace	Timestamped history or fixed latest-value rows. Resize column boundaries to reveal long Data or Signals fields. Horizontal scroll exposes FD/BRS/ESI.
D. Under trace	TX channel, message ID, Extended ID, FD frame, BRS, data, interval, Send/Stop and Multi-message TX....
E. Recording area / bottom	Logging/save/playback, recorded-bus and direction selectors, logger/playback status and CAN RX/TX/visible-row counts.

Table 12: Read the CAN window from top to bottom.

Use **View > Signal Graph** to plot decoded CAN signals and **View > CAN Bus Health** for channel statistics. See section 4.11 for plotting live traffic and replay.

Adapter defaults: CAN, LIN and Ethernet lists are alphabetical. Each protocol selects Virtual on first launch, then restores its last selection for this Windows user. Restoration does not connect. CAN/LIN save the backend family; Ethernet saves the NIC/device identity and falls back to Virtual if unavailable, preserving the saved choice for a later Refresh.

4.1 CAN columns and controls

Column	Meaning
Timestamp	Seconds with six decimal places from the message timestamp; displayed as a number, not time-of-day. Physical precision depends on the source, not the number of printed digits.
Counts	Cumulative displayed observations for channel + ID + standard/extended format since Clear. RX and local TX for that key contribute; not a dropped-frame count.
CAN Channel	Physical channel label, e.g. CAN1/CAN2, or native identifier (COM3, PCAN channel) when no label is known. Separate from the MDF Recorded bus selector.
Message ID (hex) / Ext	Arbitration ID; Ext = Y for 29-bit format, blank for 11-bit. Leading zeros do not change the ID.
Direction / DLC / Data	RX or TX (local estimate or driver report). Hover Direction/Timestamp for evidence. DLC is byte count, not the encoded FD nibble. Data is raw hex bytes.
DBC Message / Signals	Matched message name and decoded signal values/units; unknown IDs stay raw, short/invalid payloads show a decode error.
FD / BRS / ESI	Y marks CAN FD, bit-rate switch, or the received sender's error-state indication. ESI is not a bus-off indicator for the local adapter and is not user-editable TX.

Table 13: CAN trace columns in left-to-right order.

Control	Behavior
Adapter / Channel / Bitrate	Select the backend and detected device/network before Connect. Manual channel entry is available on CAN. Connection settings lock while connected.
CAN FD / Data bitrate	Connection-level FD mode and data rate. Off/unavailable on unsupported backends; frame-level FD is separate.
Driver TX reports	Opt-in before Connect for Vector/Intrepid/Kvaser/PCAN/NI-XNET. Replaces synthetic TX with driver events; not intended-ECU proof. Section 6.4.
Inject demo traffic	Virtual CAN only; generates the demo frames roughly every 100 ms.
Connect / Disconnect	Open or close the selected bus; disconnect also stops repeated sending and playback and finalizes active logging.
Clear / Refresh devices / Vector DLL	Clear resets trace/history/counters, not a saved log. Refresh enumerates devices. Vector DLL saves an XL DLL path; restart to load it.
Start Exploring / Pause Exploring	Connection starts exploring. Pause freezes trace and display counters while RX draining, TX and logging continue. Paused frames are not backfilled.
Auto-scroll (all messages)	Default unchecked: latest row per channel/ID/format. Checked: timestamped history, last 10000 retained. Toggle preserves recent history.
Apply filter / Clear filter	Apply hex ID/range expression; Enter also applies. Clear restores all IDs. A filter affects display only; invalid input keeps the previous filter.
Load / Unload / Edit DBC	Load decodes rows; Unload returns to raw. Edit opens the database editor (section 4.8); it does not transmit signal values.

Table 14: Connection, trace and decoding controls. See sections 5.2 and 5.4 for sending and recording.

4.2 LIN graphical reference map

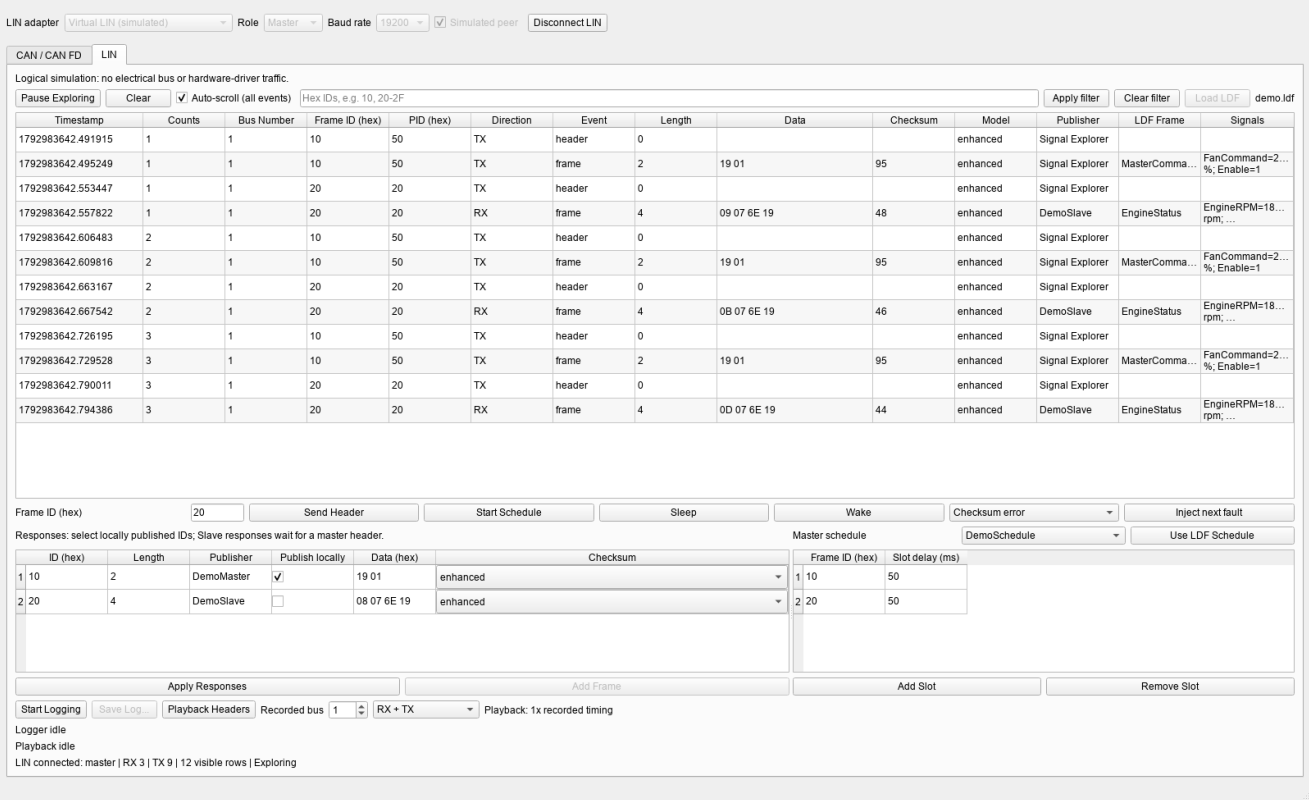


Figure 3: Virtual LIN Master after schedule traffic. The same protocol controls bind to native transports.

Region	What it contains
A. Connection bar	LIN adapter, detected channel and Refresh for native hardware; Role, Baud rate, Simulated peer for Virtual, Connect LIN/Disconnect LIN.
B. Trace toolbar	Start/Pause Exploring, Clear, Auto-scroll (all events), hex ID filter, Apply/Clear filter, Load LDF and the loaded filename.
C. Trace and direct operations	Event rows, Frame ID input, Send Header, Start/Stop Schedule, Sleep, Wake and Virtual-only Inject next fault.
D. Response table	ID, Length, Publisher, Publish locally checkbox, Data (hex), Checksum selector; Apply Responses and Add Frame.
E. Schedule / recording	LDF schedule selection, Use LDF Schedule, editable slots, Add/Remove Slot; shared MF4 logger/playback and LIN status line.

Table 15: LIN region map.

A disabled control normally reflects the selected role, disconnected state, active schedule or active playback. Changing tabs does not disconnect the other protocol. The bottom LIN status shows its own RX/TX/visible-event counts; the CAN status line is hidden while LIN is selected.

4.3 LIN columns, roles and buttons

Column(s)	Meaning
Timestamp / Counts / Bus Number	Event timestamp in seconds; cumulative count per channel + six-bit ID + event kind; physical LIN channel number. Header and response are separate events.
Frame ID (hex) / PID (hex)	Six-bit ID 00-3F versus the parity-protected byte. Enter the six-bit ID when requesting a header, not the PID. TX IDs 3E/3F are reserved.
Direction / Event	RX/TX for the observed part; event kind can be frame, header, no_response, checksum_error, parity_error, collision, sleep or wake.
Length / Data / Checksum / Model	Observed payload byte count and hex bytes, checksum when available, classic/enhanced model. IDs 3C/3D always use classic checksums; responses require 8 bytes for these diagnostic IDs.
Publisher / LDF Frame / Signals	Simulator publisher label where available, matched LDF frame and decoded signals. Native publisher identity may be blank; LIN frames do not carry a node name.

Table 16: LIN trace columns in left-to-right order.

Control	Behavior / role restriction
Role / Baud rate / Connect LIN	Monitor: no local response or header requests. Master: headers/schedules and selected local responses. Slave: buffer updates, responds to external headers. Disconnect before role/rate changes.
Start/Pause Exploring / Clear / Auto-scroll / filter	Trace behavior matches CAN; fixed LIN rows also distinguish event kind. Filter uses six-bit IDs 00-3F. Logging and schedules are independent of trace pause.
Load LDF	Disconnect first. Loads unconditional frames, signal decoding, initial values, publishers and supported schedules. No separate Unload LDF button. Demo LDF loads automatically at startup.
Frame ID / Send Header	Master only; ID must have a defined response length. Sends a request; an external or local publisher supplies data.
Start/Stop Schedule	Master only; cycles configured slots. Stop is separate from Pause Exploring. Desktop timers are approximate; slots must meet the wire-time budget.
Sleep / Wake	Master Sleep stops scheduling then requests sleep. Vector/PEAK have native Wake. Intrepid dedicated Wake is disabled. Monitor cannot send either.
Fault selector / Inject next fault	Virtual only: checksum, no response, PID parity or response collision on the next transaction. No physical error injection is offered.
Publish locally / Data / Checksum / Apply Responses	Select IDs owned locally and edit payloads; Apply updates buffers in an active Master/Slave session. Native checksum-model changes require disconnect.
Add Frame	Disconnected only; enter six-bit hex ID and decimal length, e.g. 21, 4. Adds a manual unconditional definition; it does not write an LDF file.
Use LDF Schedule / Add Slot / Remove Slot	Copy chosen LDF schedule into editable slots; add a default slot or remove the selected row. Each row is hex ID + decimal delay (ms). Not an on-disk schedule editor.

Table 17: LIN connection, editing and transmission controls.

4.4 Ethernet graphical reference map

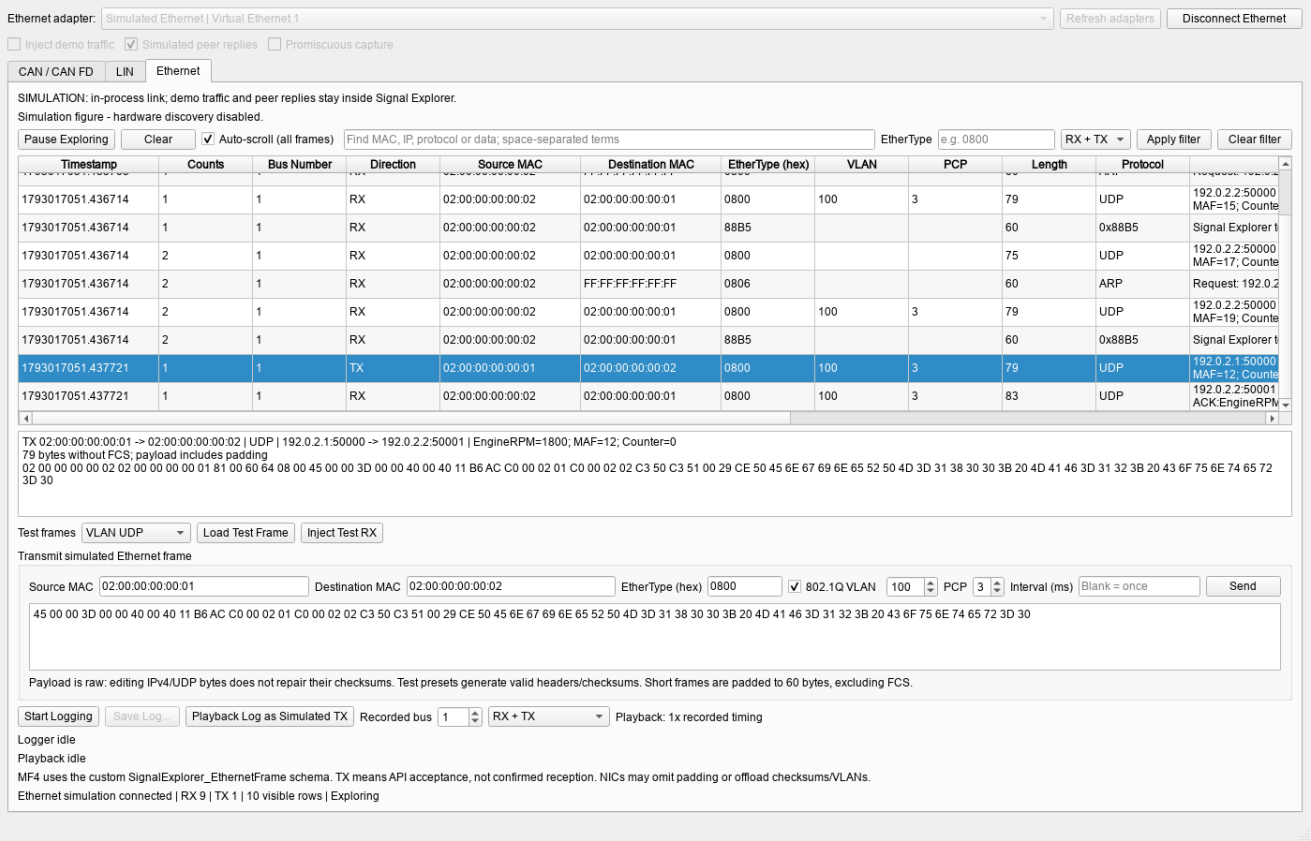


Figure 4: Ethernet tab with synthetic VLAN/UDP traffic; no physical transmission was used for this figure.

Region	What it contains
A. Above protocol tabs	Ethernet adapter selector, Refresh adapters and Connect/Disconnect Ethernet; Inject demo traffic, Simulated peer replies and Promiscuous capture options. Connection settings lock while open.
B. Notes / trace toolbar	Transport details and scan results; Start/Pause Exploring, Clear, Auto-scroll, search, EtherType, RX/TX selector and Apply/Clear filter.
C. Trace / selected frame	Chronological frames or latest flow rows. Scroll horizontally for Summary and Data. Select a row to show full raw frame bytes below the trace.
D. Test frames / transmit editor	Preset selector, Load Test Frame, Inject Test RX; source/destination MAC, EtherType, VLAN/PCP, hex payload, interval and Send/Stop.
E. Recording / status	Logging, save and playback; recorded bus/direction; logger and playback status; counts, driver drops, rejected frames and display drops on native sessions.

Table 18: Ethernet layout; complete column and editing reference follows.

The simulator is selected by default. Demo/peer/Inject Test RX apply only to it. Promiscuous capture applies only to PC Ethernet. The active tab has its own monitor and recording controls; changing tabs does not disconnect other devices.

4.5 Ethernet columns & editor

Column(s), left to right	Meaning
Timestamp / Counts / Bus Number	Unix seconds; cumulative observations per retained flow; current Ethernet bus is 1. A flow includes bus, direction, MAC pair, EtherType, VLAN and decoded IP endpoints/ports.
Direction / Source MAC / Destination MAC	RX/TX, then source and destination layer-2 addresses. PC capture TX is inferred from NIC source MAC; locally submitted TX is reported after API acceptance.
EtherType (hex) / VLAN / PCP	Inner Ethernet II type; blank VLAN/PCP for untagged frames. VLAN ID 0-4094, priority 0-7. Example 0800 = IPv4; 0806 = ARP; 88B5 = raw test.
Length / Protocol / Summary / Data (hex)	Length excludes preamble, SFD and FCS. Driver-provided padding is retained; NIC captures may omit it. Summary decodes basic IPv4/UDP/ARP; Data is bytes after EtherType, including captured padding.

Table 19: Ethernet trace columns; row selection reveals the complete MAC header and payload.

Control	Behavior
Start/Pause Exploring / Clear	Pause freezes display/counters, not capture, TX or logging. Resume has no backfill. Clear resets trace and display counts only.
Auto-scroll (all frames)	Checked: last 10000 chronological frames. Unchecked: up to 10000 latest flows. Flow counts reset on Clear or eviction; both caches are bounded.
Search / EtherType / RX + TX	Search matches all space-separated terms across MAC, IP, protocol, summary, VLAN and data hex, case-insensitively. EtherType is hex. Apply/Enter activates; invalid input keeps the old filter. Direction selector applies immediately.
Load Test Frame / Inject Test RX	Load populates the editor; PC presets use its NIC source MAC. Inject adds one synthetic RX and is simulation-only. Preset IP/destination addresses are test values, not discovered peers.
Source / Destination MAC	Six hex octets separated by colons or hyphens. Source must be nonzero and unicast. Broadcast destination is allowed.
EtherType / 802.1Q VLAN / PCP	Ethernet II type 0600-FFFF; do not enter a VLAN TPID as the inner type. Optional single tag, ID 0-4094 and PCP 0-7. No stacked tags.
Payload / Interval / Send / Stop	Raw payload hex, up to 1500 bytes. Blank interval sends once; positive integer ms repeats with Stop. Short TX is padded to 60 bytes, no FCS. Inputs lock during repeat/playback.

Table 20: Ethernet controls. Log controls follow section 5.4; procedures are in section 5.5.

Note - Editing raw IPv4/UDP payload does not repair checksums. Presets generate valid checksums; NIC offload can make outbound capture checksums appear invalid. No Ethernet signal database, TCP session editor, SOME/IP or DoIP decoder is provided.

4.6 File Viewer: Python scripts

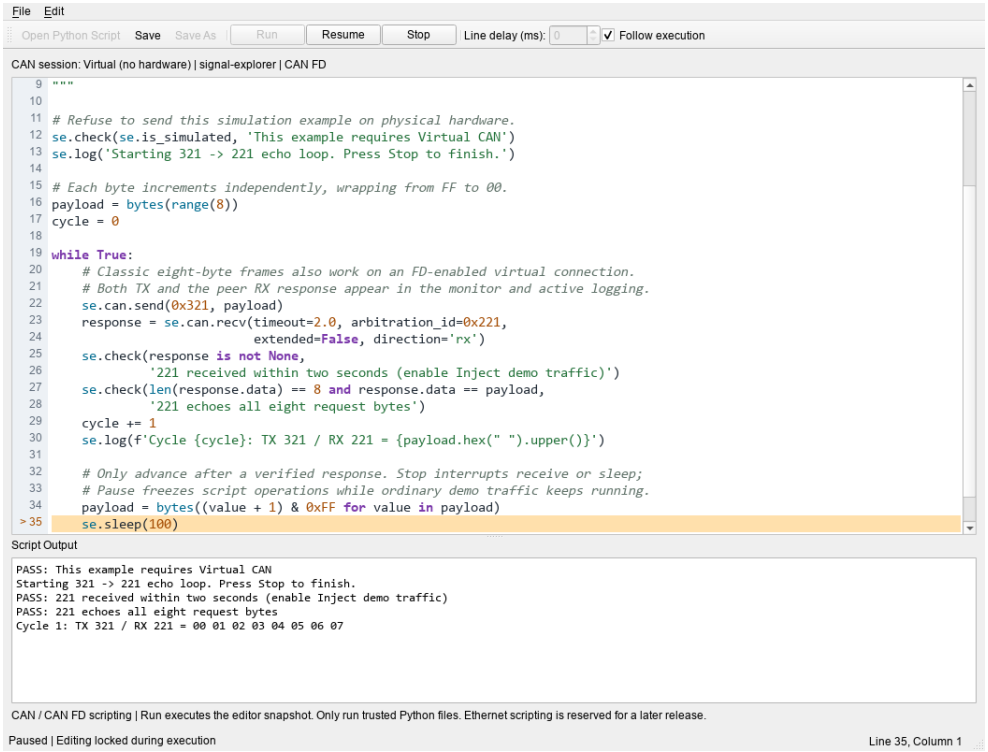


Figure: Python editor running the virtual 321/221 echo example, paused on a highlighted line.

Control	Meaning
File Viewer menu	New Python Script, Open Python Script..., Open CAN Simulation Example and Open Logfile. Python always uses the connected CAN/CAN FD session.
File / Edit	Open/New, Save/Save As, undo/redo, cut/copy/paste, Select All and case-sensitive Find/Replace. Run uses the editor snapshot, including unsaved edits.
Run / Pause / Resume / Stop	One script process at a time. Pause freezes script checkpoints and its API waits; Stop cancels further bus operations without disconnecting. Editing locks while running.
Line delay / Follow execution	Python delay: 0-1000 ms before each source line; changes actual test execution. Follow scrolls to the executing line. Yellow=running, orange=paused, red=failure.
Script Output	Print/log messages, PASS/FAIL checks and final outcome. The console retains at most 5000 text blocks. A failed check normally stops the run.

Table 21: Script-mode reference. Full API and MCU examples are in the Scripting User's Guide.

se.can.send/recv accept channel= with an enabled label/native ID; omitted TX uses the main destination and omitted RX accepts all enabled ports. se.session["channels"] lists the map; returned frame.channel is a native string. Scripts share the existing service, not another can.Bus. LIN/Ethernet scripting, CAPL conversion and full UDS/ISO-TP remain future work.

4.7 File Viewer: logfile replay

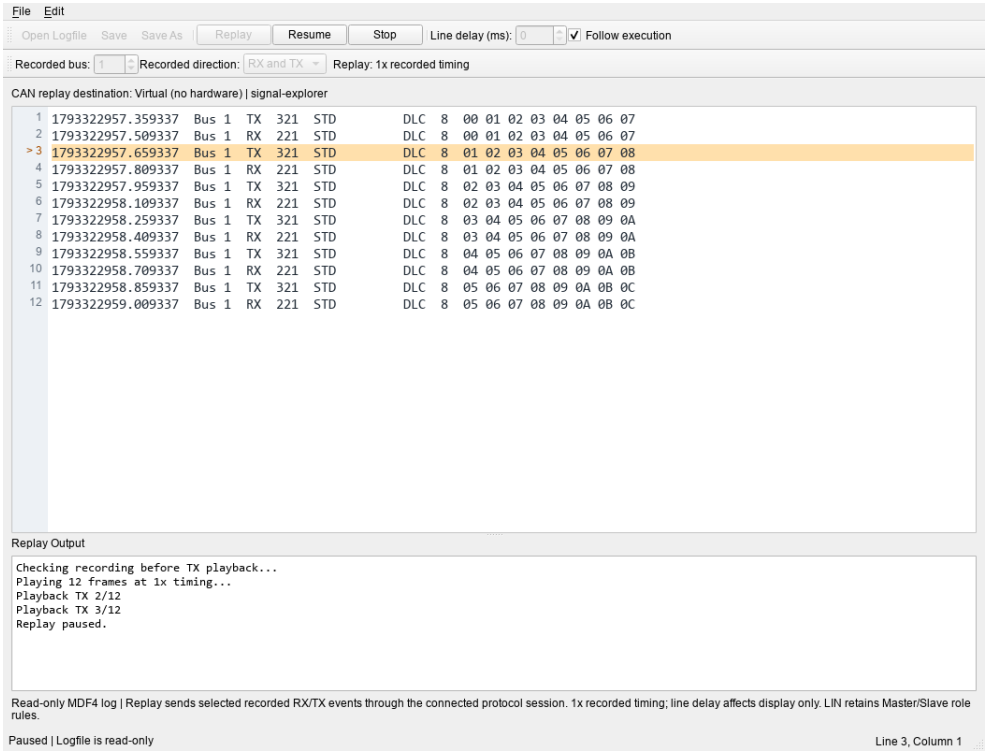


Figure: read-only log viewer paused during replay. RX/TX labels describe the original recording.

Control	Meaning
Open Logfile	Loads a single-protocol CAN/CAN FD, LIN or Signal Explorer Ethernet MF4. May be opened without a connected adapter. Contents are read-only; Save/Save As are disabled.
Recorded bus / direction	Choose source BusChannel and recorded RX, TX or both. CAN uses one main TX channel. Starting replay stops GUI cyclic senders; selected events become TX.
Replay / Pause / Resume / Stop	Validate the entire selection, then transmit at 1x timing. Pause excludes paused time from the schedule. Stop leaves the connection open; Replay restarts from the beginning.
Line delay / Follow execution	In log mode, delay controls highlight refresh only. It never delays transmission. Follow scrolls to the most recently sent original log row.
Replay Output	Load, preflight, progress, completion, failure, pause and stopping messages. The main monitor and logger remain independent.

Table 22: Log mode. Existing protocol-tab playback controls remain available.

Display cap: first 100000 rows or about 32 MiB of text; the output reports truncation, but replay reads the entire file. No ASC/BLF/PCAP import, seek, speed or automatic loop. LIN retains Master-header/Slave-buffer semantics.

4.8 DBC editor reference map

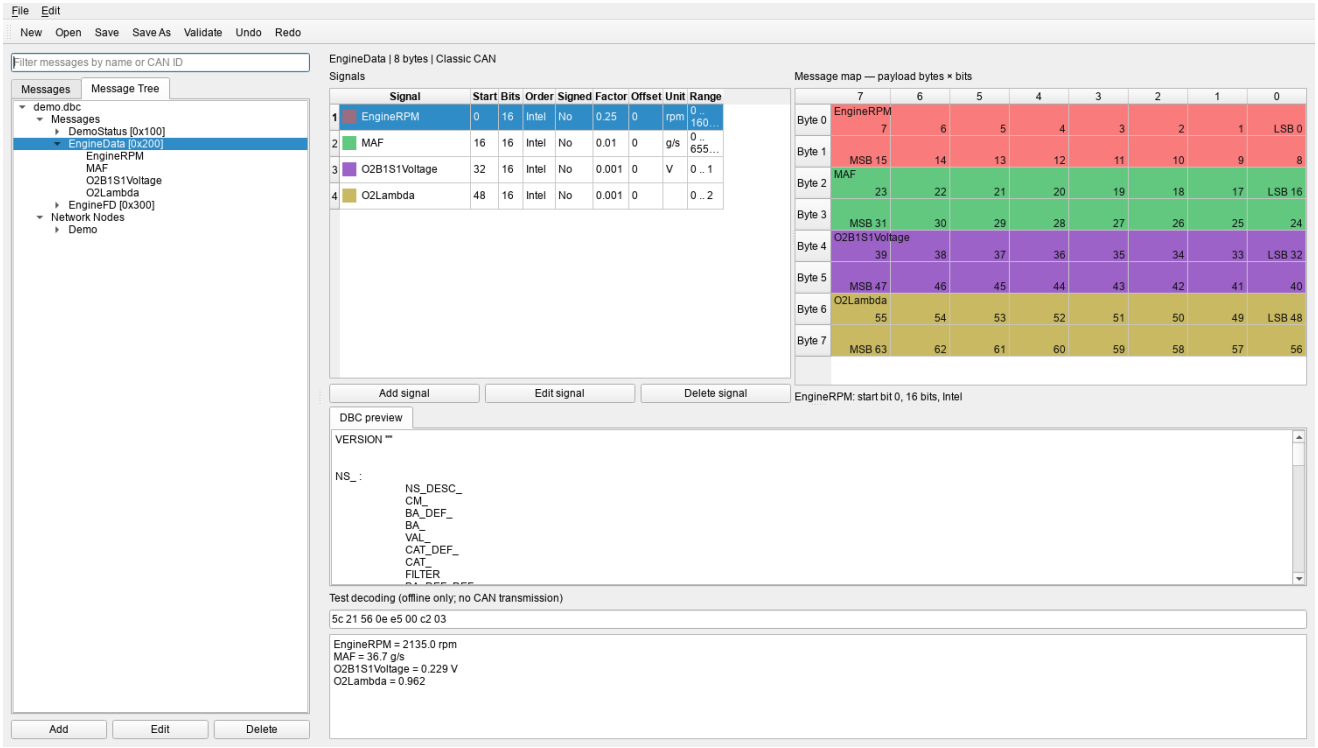


Figure: Message Tree, color-coded EngineData bits and offline payload decoding.

Region / control	Function
File / toolbar	New, Open, Save, Save As and Validate. Edit menu / toolbar: Undo and Redo. Standard keyboard shortcuts are available.
Messages, left	Messages lists hex IDs. Message Tree expands messages into signals and groups Tx/Rx messages by network node. The filter narrows both tabs as you type a name or hex/decimal CAN ID. Double-click a message or signal to edit it.
Signals, center	Name, Start, Bits, Order, Signed, Factor, Offset, Unit and Range. Add/Edit/Delete signal or double-click its row. Properties also include receivers, comments and raw-value labels.
Message map, right	Colored cells show DBC bit numbers. Signal names appear once at the top of each visible field. Wider signals show MSB and LSB; single-bit signals show just the number. Hover for full names.
DBC preview / Test decoding	Drag the divider above the preview to give the text more room. Enter exactly the selected message's byte length in hex for physical values and value labels. This calculation is offline; it sends no frame.

Table 23: Open with Edit DBC beside Load DBC / Unload DBC on the CAN screen.

The editor remembers its window size and position, divider positions and signal column widths. It uses a private database copy. Live monitoring, scripts and recording continue. Unsaved edits do not change decoding; section 5.7 explains save/reload rules and editing limits.

4.9 Multi-message sender: controls

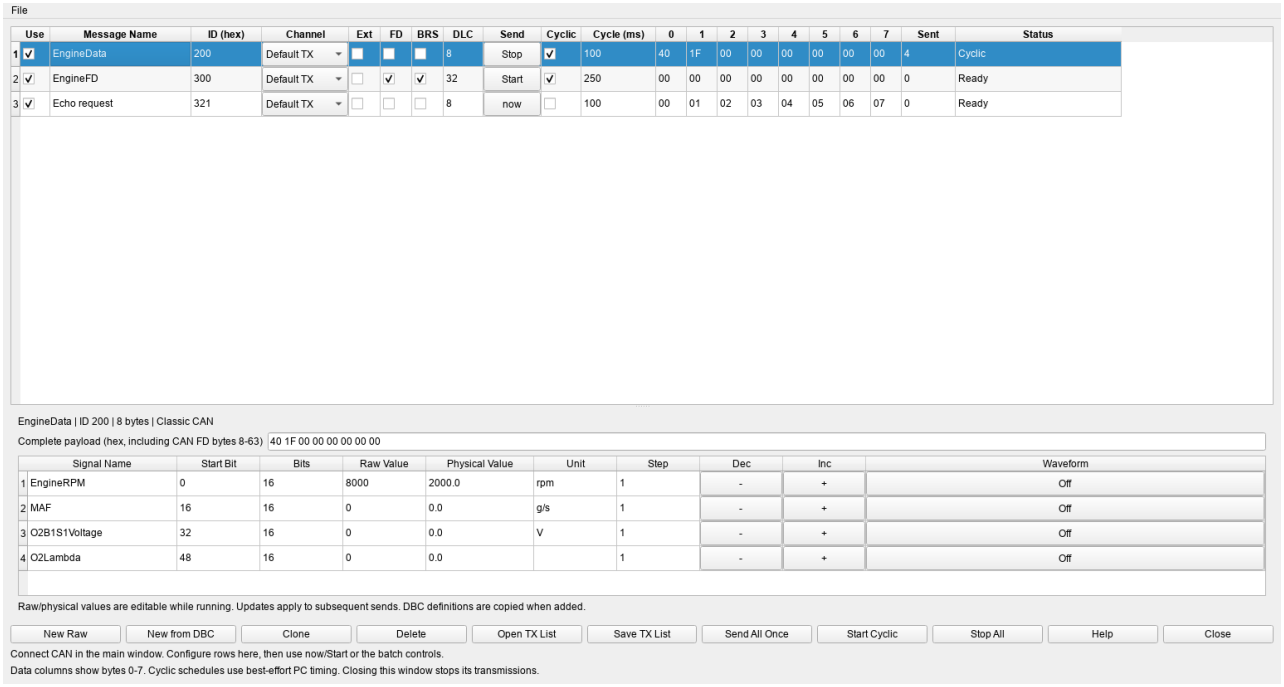


Figure: Send Messages on Virtual CAN FD; the EngineData row is sending cyclically.

Open **Transmit > Multi-message sender...** or **Multi-message TX...**. The main single-message sender remains available. This window shares the active CAN service, enabled channels, monitor and logger.

Control / columns	Meaning
Use / Message Name / ID (hex)	Use includes the row in batch actions; clearing it stops that row. Name is a label; ID is hexadecimal.
Channel / Ext / FD / BRS / DLC	One enabled destination or Default TX; standard/extended, frame-level FD and bit-rate switching. DLC is payload byte count.
Send / Cyclic / Cycle (ms)	now sends once. Check Cyclic and press Start for periodic sending; Stop ends it. Checking a box alone never starts traffic.
0-7 / Complete payload	Grid bytes are hexadecimal. Complete payload includes all FD bytes 8-63. Resize DLC first; enter exactly the required byte count.
Sent / Status	Locally accepted send count and row state/error. Acceptance is not peer ACK. One failing row stops independently.
New Raw / New from DBC	Raw frame or a private snapshot of a loaded DBC definition, including scaling, defaults and cycle metadata.
Clone / Delete / file controls	Clone selected configuration with fresh counts, or delete rows. Open TX List / Save TX List also exist under File. No cut/copy/paste controls.
Send All Once / Start Cyclic / Stop All	Batch enabled rows, start enabled cyclic rows, or cancel all queued/scheduled sends. Batch preflight precedes any TX.

Table 24: Message grid and command bar.

4.10 Signal values and live editing

DBC signal panel

Select a DBC row. Signal Name, Start Bit and Bits describe its layout. Raw Value and Physical Value are editable numeric values; Unit comes from the DBC. Step is in physical units, used by Dec/Inc. Value labels appear in tooltips. Encoding preserves bits outside active signals and validates scaling, signedness, byte order and ranges.

Waveform	Parameters and behavior
Off	Use the configured value.
Ramp	Increase by Step per cyclic transmit; wrap from maximum to minimum.
Pulse	Alternate physical minimum/maximum at 50% duty over Period (ms).
Sine	Vary between physical minimum/maximum over Period (ms).

Table 25: Waveforms advance at each row transmit, not at an independent hardware clock.

Waveforms restart when a row starts. A one-shot uses its initial waveform value. Multiplexer selectors and inactive mux signals cannot be waveform targets. Manual signal edits disable only that signal waveform; raw byte/full-payload edits disable every waveform in that row. Other existing wave phases continue across live edits.

Editing a running row

IDs, channels, Ext/FD/BRS, DLC, bytes, cycle times, signal values and waveform settings remain editable. Enter commits a table cell; leaving Complete payload commits its edited text. Refresh does not replace the field being typed in. A complete edit is validated before atomic application. Invalid changes keep the old configuration running.

Subsequent sends use the accepted edit. An already dispatched frame can finish with old values; counts continue. A cycle-time edit reschedules the next send. Default TX pins the main destination when sending starts; changing a row channel live retargets subsequent frames. Editing ID, format or DLC can detach its DBC snapshot and waveforms.

Scheduling and lifecycle

Stop All, closing this window, CAN disconnect or starting logfile playback stops its schedules. Reopening retains rows in this app session; reconnecting never restarts them. Rechecking Use/Cyclic requires Start. Monitor pause/filtering does not stop TX or exclude it from logging. Python and main-window senders are independent.

Note - Timing is best effort on the PC. Arbitration, driver latency and competing sends can delay frames; missed cycles are skipped without a catch-up burst. This window does not generate RTR/electrical error frames or provide hardware-timed scheduling.

4.11 Signal Graph

Plot live CAN signals

1. Load the matching DBC on the CAN screen and open **View > Signal Graph**.

2. Select the CAN channel and direction. Choose a DBC message and signal, then click **Add signal**.

3. Choose a 10, 30 or 60 second time window. Start or resume Exploring to collect new samples. The graph collects matching frames while its dock is open.

Overlay signals

Choose another message and signal. Under **Time window**, set **Add To** to any signal already on the graph you want, then click **Add signal**. Choose **New graph** for a separate plot. Each overlaid line has a contrasting color and a legend entry. Signals on the same plot share one vertical scale, so signals with comparable units and ranges are easiest to compare.

You can plot up to six signals. Select a signal in the list and click **Remove selected** to remove its trace. **Clear samples** empties the plots while keeping the selected signals.

Plot CAN log playback

Open the graph before starting **Playback Log as TX**. Select the playback destination CAN channel and **Sent (TX)** in the graph. Load the matching DBC and add the signals you want to see. Replayed frames must appear as TX in the CAN trace. Opening a file for offline inspection alone does not feed the graph.

> **Note** - If the graph dock is checked but missing, choose **View > Reset CAN Tool Layout**, then reopen the graph. This resets the other CAN tool dock positions as well.

5 Practical workflows: CAN and DBC

5.1 Decode a database and filter IDs

1. Connect Virtual with demo traffic or a correctly configured physical bus. Click Load DBC and choose examples/demo.dbc or the correct project database.

2. Expect ID 100 = DemoStatus, 200 = EngineData, and 300 = EngineFD only in FD demo mode. Engine signals include RPM, MAF, O2 voltage and lambda. These are examples, not OBD-II requests.

3. Enter **100, 200-2FF** in ID filter and Apply filter (or Enter). IDs are hexadecimal; spaces/commas separate entries. This example hides 300. Clear filter returns all rows.

4. Uncheck Auto-scroll to compare the latest bytes/signals by ID. Recheck to return to retained time history. Pause Exploring freezes the trace but does not stop logging or TX.

5.2 Send a frame once or every 100 ms

Field	Classic example	FD example
Connection	FD unchecked, 500 kbit/s	FD checked before Connect; 500 kbit/s arbitration, 2 Mbit/s data
Send ID (hex)	200	300
Extended ID	Unchecked for 11-bit 200	Unchecked; use it only for an actual 29-bit ID
FD frame / BRS	Both off	FD frame on; BRS on if the network uses rate switching
Data (hex)	DE AD BE EF	00 01 02 03 04 05 06 07 08 09 0A 0B (12 bytes)
Interval (ms)	Blank for once; 100 to repeat	Same interval rule
Send / Stop	Send transmits immediately, then repeats if an interval exists. Click Stop to end.	Stop before editing the payload or connection.

Table 26: Raw frame editor examples; identifiers/payloads must be suitable for the bench.

The interval must be a positive integer, not 100.5 or 0. The first send occurs immediately. Repetition locks its input fields and stops on send error, disconnect or close. Playback also stops repeated sending. This is host-timed traffic, not deterministic hardware scheduling.

> **Note** - TX indicates the local adapter accepted a send; it does not prove another CAN node acknowledged or consumed it. Verify the bytes at the peer. DBC signal encoding is available in Multi-message TX.... There is no manual RTR/error-frame generator or automatic physical-ECU response logic. The virtual 321/221 demo is a separate synthetic peer.

5.3 LIN Master and Slave workflows

Load an LDF and run as Master

1. Disconnect LIN, Load LDF, and confirm its available baud rate. The bundled demo has MasterCommand at 10 (2 bytes) and EngineStatus at 20 (4 bytes).

2. Select Master. Verify Publish locally matches your node. Demo MasterCommand starts with 19 01; EngineStatus is normally supplied by the simulated or external slave.

3. Connect LIN. Type 20 and Send Header to request EngineStatus. Select the demo schedule and Use LDF Schedule: IDs 10 and 20 have 50-ms slots.

4. Start Schedule and inspect the events. Stop Schedule ends requests; Pause Exploring only freezes display. Sleep also stops the schedule before issuing the sleep command.

Configure Slave response buffers

1. Disconnect and choose Slave. For Virtual leave Simulated peer enabled to supply the master. For native, arrange a powered external master using the same LDF.

2. Select local response IDs. For demo ID 20 enter four bytes, e.g. 08 07 6E 19, and keep the LDF checksum model. Connect; Apply Responses after any data edit.

3. Observe TX only after a matching header. In Virtual mode unchecking the local publisher can cause the demo peer to supply that ID; native tests need an intentionally quiet alternative publisher.

4. Uncheck an ID and Apply Responses to remove its responder. Verify on the external tool that no stale native response remains. This specific behavior still needs powered qualification on Intrepid firmware.

Manual frames and schedule limits

Use Add Frame while disconnected to define **21, 4**. Edit its Data and publisher checkbox. Add Slot, enter ID 21 and a suitable delay. The slot minimum uses 1.4 x nominal header/response wire time: **1.4 x (34 + 10 x (length + 1)) x 1000 / baudrate** milliseconds. At 19200 with a four-byte response that is about 6.13 ms; a 50-ms slot is a convenient demo choice.

Unsupported LDF schedule commands fail explicitly. Sporadic/event-triggered schedules, LIN diagnostic transport/NAD services and K-Line are not implemented. Changing the table does not save a new LDF. Real timing and electrical behavior must be verified separately.

> **Note** - A LIN master can also publish a response; Slave mode does not transmit complete unsolicited frames. Use exactly one master and one intended publisher for each response ID.

5.4 MF4 logging and timed playback

Record and save

1. Connect the relevant protocol and Start Logging. Both RX and TX observations are captured before display filtering/pause. A CAN log includes all enabled CAN channels; separate protocol tabs create separate files.

2. Stop Logging drains/finalizes the MDF4 recording and opens the save dialog. Enter a filename; .mf4 is appended when necessary.

3. Cancel keeps the finalized temporary recording for Save Log... and blocks starting a new log. When exiting, the app asks Save log file before exiting? Yes opens Save; No discards the temporary log and continues closing. Canceling Save during exit keeps the app open.

4. Disconnect or closing finalizes active logging too. Close waits for recording/playback work and save handling. Temporary files are not a promised power-loss recovery system.

Playback controls and meaning

Control / status	Effect
Start Logging / Stop Logging	Begins capture or finalizes it. Filtering, Clear and Pause Exploring do not erase or limit logged traffic.
Save Log...	Enabled when a finalized recording is pending; save it after cancellation or a failed destination write.
Recorded bus	Selects the file's MDF BusChannel, 0-255. CAN preserves each enabled channel: Vector CAN1-CAN4 map to buses 1-4, including sparse selections. The embedded session channel map records native IDs and labels.
RX + TX / Recorded RX only / Recorded TX only	Selects source observations for replay. Every selected replayable CAN/Ethernet frame becomes TX on the current output; recorded RX is not a passive receive.
Playback Log as TX / Stop Playback	CAN/Ethernet: preflight, then best-effort 1x relative timing through the connected output. Virtual Ethernet says Playback Log as Simulated TX. No seek, speed or loop controls.
Playback Headers / Replay Response Buffers	LIN Master replays headers with current response configuration. LIN Slave replays timed changes to selected local buffers; the external master still triggers wire TX. Monitor cannot replay.
Logger / playback status lines	Show active, finalizing, pending/saved, preflight, progress, completion, skipped events and maximum scheduling delay, or failure details.

Table 27: Shared recording panel and protocol-specific replay labels.

Use the same protocol and compatible FD mode; do not replay an arbitrary MF4 and expect unrelated signal groups to become bus frames. CAN error frames are not transmitted. Recording while replaying captures the new session's observations. RX+TX may contain related requests/responses or duplicates: choose the appropriate direction.

CAN uses the python-can MF4 schema; LIN uses LIN_Frame plus custom SignalExplorer_LINEvent events. Ethernet uses custom SignalExplorer_EthernetFrame records, with hardware/simulation, TX semantics and FCS exclusion in session metadata. Ethernet is an MDF 4.10 container, not standardized ASAM Ethernet bus-event interoperability. Round trips are not independent ASAM certification. Clock domains are not synchronized across separate adapters.

5.5 Ethernet test, capture & replay

Exercise the complete path in simulation

1. Choose Simulated Ethernet, enable Inject demo traffic and Simulated peer replies, then Connect Ethernet. Watch UDP telemetry, ARP request, VLAN UDP and raw 88B5 frames rotate every 100 ms.

2. Select **VLAN UDP** and **Load Test Frame**. Expect type 0800, VLAN 100 and PCP 3. Send with a blank interval: one TX and a synthetic RX containing ACK should appear.

3. Enter **100** in Interval (ms) and Send. The button becomes Stop; press it to end repetition. To inspect the trace without stopping the sender, use Pause Exploring.

4. Enter **EngineRPM 192.0.2.1** in search and **0800** in EtherType, then Apply filter. All terms must match. Use RX only to inspect replies; Clear filter restores all rows.

5. Start Logging, pause or filter the trace, and send a few test frames. Stop Logging and save an .mf4 file. Both RX/TX are recorded despite the display state.

Replay into simulation, then a controlled hardware bench

1. Disconnect; uncheck Inject demo traffic and Simulated peer replies before reconnecting for a replay-only run. Choose Recorded bus 1 and the intended recorded direction.

2. Click Playback Log as Simulated TX and choose the Ethernet log. Selected RX and TX both become transmitted frames at 1x relative timing. Stop Playback interrupts remaining sends; starting playback stops repeated Send.

3. For hardware, disconnect and choose the intended PC port or powered ValueCAN4-2EL. Demo/peer controls become disabled. Verify the link and destination with an independent peer tool.

4. Load/edit a test preset for that network and Send once. Preset addresses 192.0.2.1/192.0.2.2 and 02:00:00:00:00:01/02 are examples; they do not discover or address your real ECU automatically.

5. After peer verification, log/replay a suitable bench capture using Playback Log as TX. Replay preserves recorded source/destination bytes and adds minimum TX padding when a NIC capture omitted it. It does not rewrite MAC/IP addresses for the selected hardware.

Recording compatibility and boundaries

The Ethernet schema stores BusChannel, Direction, Length and RawBytes. The valid bytes are selected by Length; existing simulation MF4 files still replay. CAN/LIN logs and arbitrary signal-only MF4 files are rejected by Ethernet playback. Hardware send success means driver/API acceptance, not received bytes at a peer.

> **Note** - The simulator's ACK is a synthetic response to valid UDP or raw 88B5 sent to its peer. Hardware has no such automatic reply. No PCAP import/export, auto-ARP or TCP/IP session stack is implemented. Desktop timing is best-effort, not hardware scheduling.

5.6 Python CAN and MCU test workflow

Start with the built-in virtual peer

1. Connect Virtual CAN with Inject demo traffic enabled. Optionally enable CAN FD before connecting.

2. Choose **File Viewer > Open CAN Simulation Example** and Run. Expect TX 321 with 00 01 02 03 04 05 06 07, then RX 221 with the same eight bytes.

3. After each matching reply the script increments every byte, wrapping FF to 00, and waits 100 ms before the next request. The loop ends only on Stop or a failed check/error.

4. Pause/Resume in the editor to inspect a line while normal demo traffic continues. Stop keeps the adapter connected. Monitor pause/filter does not stop the script.

5. Start Logging before running to capture both sides. Stop Logging and save, then use File Viewer > Open Logfile to inspect or replay a chosen direction.

Move to your MCU firmware contract

Define IDs, standard/extended format, byte order, command/response layout, sequence fields, response deadlines and expected values before changing a script. The built-in simulator implements the 321/221 echo and periodic demo traffic; it is not a generic MCU, UDS server or hardware fault injector.

The companion Scripting User's Guide includes GPIO/ADC, reboot, negative-command and limited single-frame UDS templates. Their example IDs are fictional. They refuse TX until FIRMWARE_CONTRACT_CONFIRMED is set after reviewing the matching firmware and fixture. Source files are in examples/scripts/mcu/.

Timing and result interpretation

Use receive timeouts in seconds and se.sleep in milliseconds. Leave Python line delay at zero for timing tests. A positive local TX entry is API acceptance; verify a matching received payload or an independent physical peer.

A stopped script does not restore MCU outputs, cancel already accepted frames or reset the ECU. Use a fixture cleanup step and firmware timeout for output-control tests. Normal Python exceptions fail the run; checks already marked FAIL remain failures even if the script catches their exception.

Open and replay a saved test

1. Open the correct protocol MF4 via File Viewer, connect the intended destination and select Recorded bus plus recorded direction.

2. Use Replay, Pause/Resume and Stop in the viewer. The main tab's Stop Playback can also stop it. Only one replay per protocol can run.

3. Use Follow execution for row tracking. Log-mode line delay changes highlighting only; selected recorded gaps remain unchanged.

5.7 Create, edit and verify a DBC

Open a private editing copy

1. Load the intended DBC on the CAN screen, then click **Edit DBC**. The editor starts with a private copy of the loaded database. With no loaded database it starts empty; Open chooses another file and New starts a blank document.

2. Use the Messages list or Message Tree to select a frame. Type in Filter to narrow both tabs. Use Add/Edit/Delete below the tabs. Enter the arbitration ID in hex, select standard/extended and Classic/FD, then set payload byte length, cycle time, senders and optional comment. Cycle time is database metadata; it does not start repeated sending.

3. Use Add signal or double-click a signal row. Set name, start bit, bit length, byte order, signedness, factor/offset, optional range and unit. Receivers, comment and raw-value labels are in the properties dialog. Enter labels one per line, such as 0=Released and 1=Pressed.

4. Select a signal and check the message map. Intel uses little-endian ordering. Motorola uses DBC sawtooth numbering: start bit 7 runs through bit 0, then bit 15. Map column labels are bit positions within each byte.

5. Click Validate. Overlapping/out-of-payload signals and other invalid edits are rejected without replacing the document. Undo/Redo restores accepted edits. Test decoding with a payload of exactly the message's length to check scaling and labels offline.

Example: verify an RPM scaling definition

In an empty database add standard Classic CAN message **EngineTest**, ID **321**, length **8**. Add **EngineRPM**, start **0**, length **16**, Intel, unsigned, factor **0.25**, offset **0**, unit **rpm**. Leave optional range blank. The payload **40 1F 00 00 00 00 00 00** has raw value 8000 and should decode to **2000 rpm**. This does not send CAN traffic.

Save, reload and close

Save validates and atomically writes the DBC. If that exact file is currently loaded on the CAN screen, a successful save reloads its decoder for existing/new trace rows. **Save As** to another path does not switch the active decoder. With no active DBC, use **Load DBC** after saving. New/Open/Close prompt to save dirty work; Cancel leaves the editor open.

Current editing limits

Multiplexing and floating-point definitions are preserved but not configured by the basic forms. A multiplexer cannot be renamed, or deleted while dependents remain. Saving uses cantools serialization, so whitespace and ordering may change; text must fit Windows-1252. For live DBC signal encoding use Multi-message TX...; the DBC editor itself remains an offline definition/decoding tool.

5.8 Build, edit and save a TX list

1. Connect **Virtual (no hardware)**, enabling CAN FD before connection if needed. Load examples/demo.dbc. Open Multi-message TX....

2. Choose New Raw. Set ID 321 and eight bytes 00 01 02 03 04 05 06 07. Leave Default TX and click now. With demo traffic enabled, the simulated peer returns ID 221 echoing those bytes.

3. Choose New from DBC and select EngineData (ID 200). Select its row and set EngineRPM Physical Value to 2000; scaling encodes raw 8000. Set Cycle (ms) to 100, check Cyclic, then press Start.

4. While it runs, change EngineRPM or cycle time and commit. Inspect subsequent trace frames. In Virtual mode the demo also publishes ID 200: use Auto-scroll history to distinguish observations by Direction.

5. Optionally configure a signal Ramp, Pulse or Sine within DBC ranges. Editing that signal manually disables its waveform; editing raw payload disables all row waveforms. Stop the row or Stop All when finished.

6. Click Save TX List and choose a .json name. Close/reopen the sender or use Open TX List. Loading stops prior schedules and restores configuration only; explicitly start the desired rows.

Which file belongs to which task?

File	Purpose
DBC	Message layout, scaling and optional cycle metadata; not a complete configured transmission recipe.
Signal Explorer TX list (.json)	Versioned application format: rows, payloads, channel selections, flags, periods, Use/Cyclic, waveforms and embedded DBC definitions. Counts and running state are not restored. Not a universal vendor interchange format.
MF4	Recorded RX/TX evidence and timed replay. Replay is selected recorded bus to one selected TX channel, not a live editable multi-message recipe.

Table 28: Save configuration separately from test evidence.

A list accepts up to 256 rows; loading is bounded to 16 MiB and validates the complete document before replacement. Saving replaces the file atomically. DBC snapshots travel with the list; loading a later main DBC does not alter existing rows. Re-add a row from the updated DBC for a changed layout.

6 Troubleshooting: CAN and drivers

Symptom / fault	Interpretation and action
Connected; no RX	First verify another node is transmitting. Match channel and bitrate, check the exact connector map/power, then Clear filter and Start Exploring. USB alone creates no CAN traffic.
TX increases; peer sees nothing / ACK error	Local TX acceptance is not physical success. Check a second active CAN peer, correct termination and a compatible mode/rate. A silent/listen-only peer may not acknowledge.
Stuff error / bit, form or CRC error	A bitstream/protocol rule was violated. Check bitrate/sample timing, FD/BRS/data-rate compatibility, H/L polarity, wiring quality, stubs, ground reference and termination. Use a vendor analyzer for fault subtype.
Error passive / bus-off	The controller has confined itself after repeated errors. Stop repeat/playback, inspect the cause and recover using the vendor procedure; then disconnect/reconnect as appropriate. No dedicated reset/bus-off recovery control exists in this GUI.
CAN FD send rejected	Connect in FD mode and select FD frame; use a supported payload size. Data rate must be at least arbitration rate. The actual hardware/firmware can still reject the combination.
Known unsupported FD channel	Leaf Light R v2 and ValueCAN3 are Classic-only. Some detected channels explicitly report no FD capability. Do not infer FD support from a vendor dropdown name.
Access denied / port in use	Close the other serial app or incompatible vendor session. For Vector/Intrepid, disconnect this app's other protocol session on the same physical device.
Missing DLL / empty device list	Check 64-bit runtime and USB driver separately. Use Refresh after hotplug; restart after changing DLL paths. An installed API with no hardware can legitimately list no channels.
DBC decode error / raw-only ID	Confirm database version, standard vs extended ID and correct payload length. A decode error does not imply the physical frame had a CRC error. Unmatched IDs remain raw.
Receive failed / Send failed	Keep the entire error text including native code. A send error stops repetition; a receive failure needs attention and may disconnect the session. Correct the hardware/API issue before retrying.

Table 29: CAN and connection recovery. Error taxonomy reference [F1].

> **Note** - The CAN trace does not expose an error-type column, live controller error counters, bus-load graph or normalized vendor fault-code dashboard. A displayed ESI bit describes the sender of that FD frame, not proof that the local controller is bus-off. Inspect driver diagnostics/LEDs and a vendor console for controller state.

Disconnect power before resistance measurements. Normal CAN operation in Signal Explorer is not a selectable listen-only mode. The hardware may acknowledge traffic even when Exploring is paused.

6.1 LIN, logging and overload faults

Event / message	What to check or do
no_response	No complete response followed the header. Check publisher selection, power/reference, ID/length and a running master. A native timeout is not a valid checksum.
checksum_error / parity_error	Check classic/enhanced model and protected-ID handling. Enter a six-bit ID; do not type PID into Frame ID. IDs 3C/3D require classic checksum.
collision	More than one response publisher or a TX/RX mismatch. Ensure one master and the intended publisher per ID. Virtual injection is an example, not physical fault generation.
Vector access denied / XL error	Other software may hold initialization access; release that physical LIN channel. Check installed LIN piggy, active LIN capability and bus setup. Raw XL status is retained in error text.
PEAK PLIN error 0x....	Check PLIN Manager, PLinApi.dll, channel ownership and supply. Codes belong to PLIN, not PCAN-Basic. Consult [H10] with the exact function/code.
Intrepid receive/API errors	Capture stops on reported API errors. Verify device driver/firmware, LIN supply and the 2EL model. Close other clients; check neoVI Explorer after app disconnect.
LIN cleanup failed	Read the complete error. The app attempts remaining cleanup. Stop the external master and inspect the vendor device state before reuse; do not assume failed cleanup removed every responder.
Recording queue overflow	The 100000-event writer queue filled. Capture stops and a finalized partial log is marked incomplete. Save it, reduce offered load and check disk/CPU throughput; display filters do not reduce capture load.
Save failed - log retained	Choose a writable location with space and a different file if the source is playing. Use Save Log...; do not discard the retained temporary file before saving.
No replayable events / preflight error	Check file protocol/schema, Recorded bus, direction and selected LIN response IDs. FD or length/model incompatibility prevents replay; no selected TX should occur before successful preflight.

Table 30: LIN/recording symptoms and recovery.

Native codes are not universal

Vector XL status 10 and PLIN return 3 mean an empty receive queue in their respective APIs; the adapters consume these normally. Other numeric codes must be interpreted in the matching API manual, including the failing function name. Intrepid LIN flag examples used by this build: 0x04000000 = ID parity, 0x02000000 = TX/RX mismatch, 0x40000000 = receive timeout, 0x80000000 = no slave data (StatusBitField2). These are bit masks, not OS error numbers.

Trace retention of 10000 rows and paused display are intentional limits, not evidence of transport drops. There is no universal dropped-frame counter. Driver overrun warnings, explicit logger overflow and replay scheduling-delay messages provide different evidence; save the messages with the capture. Native LIN polling errors/overruns stop the session.

6.2 Ethernet capture & fault recovery

Symptom / status	Interpretation and action
PC port listed; capture unavailable	Npcap is missing, inaccessible or cannot enumerate that interface. Install current Npcap, then Refresh adapters. Admin-only installs may need elevation. Do not copy an arbitrary wpcap.dll into the project.
No PC Ethernet ports listed	This release lists Windows Ethernet-type interfaces only, including virtual ones. Wi-Fi/loopback are excluded. Check Device Manager and Windows network adapters; reconnect/enable the port and rescan.
Npcap open failed / permissions	Check installed Npcap service/driver and the selected adapter. Use the installer's capture permissions policy; run elevated only if required. Refresh after adapter removal or replacement.
DLT_EN10MB error	The capture interface returned a non-Ethernet link-layer format. Select a wired Ethernet interface; raw Wi-Fi/radiotap and loopback captures are not accepted.
Session open; no traffic	An open handle is not link verification. Check link LEDs/status, physical cable and a known transmitting peer. ValueCAN needs external 12 V. Clear filters and Start Exploring. Capture cannot see traffic hidden behind other switch ports without a mirror/TAP.
ValueCAN device already open / in use	Disconnect CAN or LIN on that serial and release Vehicle Spy/neoVI Explorer. A shared native multi-protocol handle is not implemented. Different devices may be used independently.
Driver drops	Npcap cumulative buffer/interface drops. Reduce incoming load and investigate capture/PC performance. Intrepid shows n/a because its binding does not expose this cumulative statistic; n/a is not zero loss.
Display drops	The bounded 2000-frame UI queue overflowed. Display missed events; the logger is fed separately. Check logger overflow messages separately rather than assuming the MF4 also lost those frames.
Unsupported/error frames	Rejected captures are not logged: e.g. truncation, jumbo payload, stacked VLAN, 802.3 length frames, invalid source MAC or Intrepid CRC/length errors. This build supports Ethernet II, MTU 1500 and one VLAN tag.
Bad IPv4/UDP checksum summary	Raw edits can invalidate a checksum. Load a fresh preset or repair the packet bytes. Outbound NIC offload may also show incomplete checksums in host capture; verify at an independent peer.
Intrepid RX/API or TX error	The session stops and logging finalizes; repeat and replay stop. Preserve the function/status text. Check power, link and PHY setup, release the device and reconnect after correcting the cause.
No replayable Ethernet events	Choose a Signal Explorer Ethernet MF4, Recorded bus 1 and a direction present in the log. CAN/LIN or unrelated MF4 groups do not become Ethernet frames. Hardware and older simulation logs share this custom schema.

Table 31: Ethernet symptoms. Native error codes are API-specific, not universal bus fault codes.

Physical Ethernet errors are not CAN stuff-errors or bus-off. CRC/length rejections, native API errors, driver buffer loss, display queue loss and recorder queue overflow are distinct conditions. No line-rate lossless performance or independent ASAM Ethernet interchange qualification is claimed.

6.3 Timestamp and script recovery

Why an older Intrepid log can jump by years

The October 23 test log recorded Intrepid RX close to a 2007-based device clock while local TX used the PC's 2026 clock. At row 4936 the saved time jumped forward by 625204447.174 seconds (about 19.8 years). Chronological correction then clamped many older RX timestamps forward; 28656 corrections were recorded in that file's session metadata. The original intervals cannot be reconstructed from the flattened timestamps.

Correction in this build

On each Intrepid CAN, LIN or Ethernet connection, the first hardware RX timestamp is anchored to PC receipt time. Later RX events retain their hardware time differences, using the same host-epoch basis as local TX. Reconnection creates a fresh anchor; a backwards device-clock step over one second re-anchors. This includes initial driver latency and does not synchronize multiple adapters or provide PTP/GPS accuracy.

Restart the application and make a fresh recording. Existing MF4 files are not rewritten. Preflight rejects selected gaps above 24 hours with the original row and gap before any replay TX. Bounded platform waits prevent the Windows timeout-overflow error. There is no Skip gaps control and valid gaps are not compressed.

Symptom	Action
This example requires Virtual CAN	Choose Virtual, Inject demo traffic and Connect, or use a reviewed hardware-specific test instead of the built-in demo.
No 221 reply / test timeout	Enable demo traffic for 321/221. For a physical MCU, check ID, format, payload contract, rate, channel and peer response. Local TX alone is insufficient.
Receive mailbox overflow	The independent 2000-frame ring overwrites oldest data and warns without automatically stopping. Use se.can.stats(), flush() and set_filters(); verify dropped counts before accepting results.
Pause remains Pausing	A blocking native call must return to a Python checkpoint. Prefer se.sleep and se.can.recv over time.sleep or direct driver calls.
Stop or disconnect	Further script bus operations are rejected. Stop is cooperative, then the worker may be killed after one second. Finally blocks are not guaranteed; inspect MCU state separately.
Logfile Replay unavailable	Wait for loading; connect the matching protocol, select a replay-capable LIN role if applicable, and stop any other playback for that protocol.

Table 32: Scripting and timestamp recovery.

6.4 CAN clocks and driver TX evidence

Before Connect, optionally select **Use driver TX reports**. It defaults off, locks while connected and is unavailable on Virtual CAN, CANable SLCAN and gs_usb. Native mode publishes only received driver TX events, with no synthetic echo or fallback confirmation. Sent counts accepted API calls; TX trace/log counts observed reports, so they can differ.

Backend	Driver-report behavior
Vector XL	Classic TX_COMPLETED and CAN FD TX_OK notifications.
Intrepid neoVI	Transmit reports without GLOBAL_ERR; CAN RX/TX share one device-clock anchor. The backend send itself can wait for a report.
Kvaser CANlib	LOCAL_TXACK with separate read/write handles, configured automatically. Other local handles can contribute TX reports.
PEAK PCAN	PCAN_MESSAGE_ECHO. PEAK documents PCAN-Basic 4.6+ and driver 4.3+ for this capability.
NI-XNET	Input-stream transmit echo enabled; echoed frames are TX.

Table 33: Compatible drivers/hardware are required. Disable this option if connection rejects echo support.

Timestamp sources and script timing

CAN host estimates use Unix time anchored once to monotonic time. Virtual/SLCAN RX uses host receipt time. Intrepid RX and TX reports preserve native intervals with one first-event anchor. Other physical CAN backends retain python-can driver epoch timestamps. Neither first-event anchoring nor a driver report synchronizes separate adapters or proves intended-ECU receipt.

Hover Timestamp/Direction for source, native timestamp and TX evidence. CanFrame adds timestamp_source, raw_timestamp and tx_confirmation. send always returns host acceptance time; use recv(direction="tx") for a driver event, allowing TX in script filters. Correlate channel, ID, payload and transaction; reports are not unique per-send receipts for this application.

Recording diagnostics

The shared recorder reorders up to 50 ms of arrival skew within bounded memory. Valid times remain unchanged. Late arrivals beyond the window or invalid times can still require clamping. Session metadata separates reordered frames from corrections, counts reasons and keeps up to 100 original/corrected/native samples plus CAN source/evidence counts. It does not preserve every raw native timestamp as an MDF channel or repair older files.

Physical driver/timing qualification is pending. Backend sources and detailed limits: docs/timestamps.md. LIN/Ethernet transport clock policies and runtime dependencies are unchanged.

7 Appendix: CANable, Kvaser and PEAK

Use the connector's molded pin numbers and the vendor view orientation. Male/female mating faces and solder faces are mirrored. Do not assign power to an unused pin from a generic CAN diagram. Tables below apply only to the named hardware; custom cables/board clones override them.

7.1 CANable / CANable 2.0 / Pro / candleLight

Terminal marking	Connection / restriction
CANH / H	Bus CAN-H.
CANL / L	Bus CAN-L.
GND	Bus-side reference per the board's isolation arrangement.
5V, if fitted	Output only on original CANable / 2.0; leave unconnected unless intentionally powering a compatible 5-V target. Never apply vehicle battery voltage here.
Termination switch / jumper	Enable only if the board is at a CAN bus end and the network needs that terminator. Position/order varies by board revision.

Table 34: Label-based terminal map; there is no universal numbered CANable connector. [H1]

7.2 Nine-pin D-Sub: Kvaser and PCAN-USB Pro FD

Pin	U100-C / Leaf Light R v2	PEAK PCAN-USB Pro FD
1	NC	NC / optional +5-V output configuration
2	CAN-L	CAN-L
3	GND	CAN-GND
4	NC	LIN
5	Shield	LIN-GND
6	NC	LIN-GND
7	CAN-H	CAN-H
8	NC	Not assigned
9	NC	LIN VBAT (8-18 V input, per LIN connector)

Table 35: Model-specific connector mapping. [H7,H8,H9]

Kvaser U100-C and Leaf Light R v2 have no built-in CAN termination. On PEAK the two combined CAN/LIN connectors repeat this mapping. Do not treat PEAK pin 5/6 LIN ground as a generic CAN shield. [H7,H8,H9]

Note - Firmware/backend changes do not change CANable terminal identities. gs_usb is a software interface, not a connector standard. Other candleLight boards require their own silkscreen/manual.

7.3 Vector connector pinouts

VN1610: dual CAN connector before a Y splitter

Pin	VN1610 CH1/CH2 combined D-Sub
1	CH2 CAN-L
2	CH1 CAN-L
3	CH1 bus ground
4	CH2 CAN-L duplicate shown in the connector/cable map; not LIN
5	Shield
6	CH2 bus ground
7	CH1 CAN-H
8	CH2 CAN-H
9	No bus signal shown / leave unconnected

Table 36: Use the exact revision and Vector CANcable 2Y mapping. CH2 branch maps device 1/6/8 to branch 2/3/7. [H5]

VN1640 / VN1640A dedicated bus sockets

Pin	High-speed CAN example: 1041Amag	LIN example: 7269mag
1	Reserved / leave open	Reserved / leave open
2	CAN-L	Reserved / leave open
3	VB- (bus reference)	VB- (LIN reference)
4	Split function; not generic CAN-L	Pdis: power-disable control
5	Shield	Shield
6	Reserved / leave open	Reserved / leave open
7	CAN-H	LIN bus
8	Reserved / leave open	Reserved / leave open
9	VB+ per piggy supply rules	VB+ per piggy supply rules

Table 37: Conditional piggy examples, not an assertion about the user's uninspected piggy model. [H6]

For VN1630/VN1630A shared CH1/CH3 or CH2/CH4 sockets: with the documented piggy configuration, secondary built-in CAN uses 1 = CAN-L, 6 = GND, 8 = CAN-H; the primary uses piggy-defined pins. Without a piggy, documented built-in CAN uses 2 = CAN-L, 3 = GND, 7 = CAN-H. DIP settings and revision determine routing. [H5]

Note - Read the actual transceiver name in Vector Hardware Config/Manager before wiring. Other CAN/LIN piggies can have different power, isolation and auxiliary pins. For 7269mag, Pdis-to-VB- disables internal supply; apply external supply only according to its manual. CH5 IO and the sync connector are not CAN/LIN bus connectors. [H6]

7.4 Intrepid CAN-family nine-pin maps

Pin	ValueCAN3	ValueCAN4-1	ValueCAN4-2
1	Trigger	NC	NC
2	CAN1-L	CAN1-L	CAN1-L
3	GND	GND	GND
4	CAN2-L	NC	CAN2-L
5	GND / shield	GND / shield	GND / shield
6	GND	GND	GND
7	CAN1-H	CAN1-H	CAN1-H
8	CAN2-H	NC	CAN2-H
9	V+ optional supply; see model guide	Unassigned (-)	Unassigned (-)

Table 38: Intrepid model-specific D-Sub; ValueCAN3 pin 9 is not equivalent to ValueCAN4-2. [H3,H4]

ValueCAN4-4 with four separate DB9 branches

For the standard four-DB9 branch cable, each labeled CAN branch uses 2 = CAN-L, 3 = GND and 7 = CAN-H. The molded branch identifies CAN1, CAN2, CAN3 or CAN4. Leave other contacts alone unless the exact cable pinout assigns them. A DB26 or OBD version is a different harness, not this four-DB9 map. [H11]

ValueCAN4-4 signal	DB26HD contact(s)	OBD-II contact(s)
CAN1-L / CAN1-H	4 / 14	14 / 6
CAN2-L / CAN2-H	6 / 16	11 / 3
CAN3-L / CAN3-H	7 / 17	13 / 12
CAN4-L / CAN4-H	2 / 12	9 / 1
Ground / supply	Not assigned in the cited diagram; verify cable	4 = CHS GND; supply not assigned

Table 39: ValueCAN4-4 alternative DB26HD [H11D] and OBD-II [H11E] maps. Unlisted contacts have no assignment in those source diagrams.

ValueCAN / neoVI is a backend family, not one connector

The GUI accepts Intrepid network names such as HSCAN and HSCAN2 (CAN1 and CAN2 on ValueCAN4-2). It may accept other neoVI hardware through python-can, but that does not make these tables valid for every neoVI cable. For another model, use the manufacturer's serial/model/cable-specific map and verify its capabilities first.

Note - The ValueCAN4-2EL connector is not the ValueCAN4-2 dual-CAN DB9. Its DB26, OBD-II and MULTI variants are on the following pages. A generic dual-CAN splitter can put a signal on a LIN or power contact when used with the wrong adapter.

Virtual CAN and Virtual LIN have no physical pins. Custom/remapped cables and unspecified piggies require their exact manufacturer map.

7.5 ValueCAN4-2EL: DB26 and OBD-II

The two columns below are different cable products, not a one-to-one adapter wiring instruction. Dash means outside the OBD connector pin count. The vendor tables leave some DB26 contacts blank; those are shown as unassigned. Ethernet is now selectable; use the correct breakout and external 12 V. The app does not control ETH ACTIVATE. [H11]

Pin	2EL DB26HD variant	2EL OBD-II variant
1	ETH TX+	CAN2-H
2	Unassigned	NC
3	Unassigned	ETH TX+
4	CAN1-L	NC
5	Unassigned	GND
6	CAN2-L	CAN1-H
7	Unassigned	ISO9141/K/LIN
8	Unassigned	ETH ACTIVATE
9	ETH TX-	CAN2-L
10	GND	NC
11	ETH RX+	ETH TX-
12	Unassigned	ETH RX+
13	Unassigned	ETH RX-
14	CAN1-H	CAN1-L
15	Unassigned	NC
16	CAN2-H	VBAT
17	Unassigned	-
18	Unassigned	-
19	VBAT	-
20	ETH RX-	-
21	Unassigned	-
22	ISO9141/K/LIN	-
23	Unassigned	-
24	Unassigned	-
25	ETH ACTIVATE	-
26	Unassigned	-

Table 40: ValueCAN4-2ELA/C-DB26 [H11A] and ValueCAN4-2ELA/C-OBD [H11B] connector maps.

Note - Standard 2EL LIN supply examples: DB26 pin 19 VBAT with pin 10 GND, LIN on 22; or OBD-II pin 16 VBAT with pin 5 GND, LIN on 7. Use the vendor-specified 12-V supply and correct cable variant. Do not infer a universal OBD diagnostic wiring policy from these vendor-defined optional pins. [H2,H11]

7.6 ValueCAN4-2EL MULTI harness

The MULTI product has distinct DB9, DB25 and OBD-II branches. The DB9 branch carries LIN, not the standard CAN pins. Blank contacts in the vendor diagram are shown as unassigned. Follow the labeled branch and exact harness part number. [H11]

Pin	MULTI DB9	MULTI DB25	MULTI OBD-II
1	ISO9141/K/LIN	Unassigned	NC
2	Unassigned	Unassigned	NC
3	Unassigned	Unassigned	ETH TX+
4	Unassigned	Unassigned	NC
5	GND / SHIELD	Unassigned	GND
6	Unassigned	Unassigned	NC
7	Unassigned	Unassigned	NC
8	Unassigned	Unassigned	ETH ACTIVATE
9	Unassigned	Unassigned	NC
10	-	Unassigned	NC
11	-	Unassigned	ETH TX-
12	-	Unassigned	ETH RX+
13	-	GND	ETH RX-
14	-	CAN1-H	NC
15	-	CAN1-L	NC
16	-	CAN2-H	VBAT
17	-	CAN2-L	-
18	-	Unassigned	-
19	-	Unassigned	-
20	-	Unassigned	-
21	-	Unassigned	-
22	-	Unassigned	-
23	-	Unassigned	-
24	-	Unassigned	-
25	-	VBAT	-

Table 41: VCAN4-2ELA/C-MULTI branch tables [H11C]. Dash = outside that connector's pin count.

Note - Unlike the dedicated OBD-II 2EL cable, the MULTI OBD-II branch does not carry CAN or LIN on pins 6/7/14 in the published diagram. Use its DB25 CAN branch and DB9 LIN branch. Check continuity with power disconnected if a custom harness or pin-remapping option is present.

7.7 NI USB-8502 connector reference

Use molded pin numbers on each 9-pin male D-Sub. Port 1 and Port 2 use the same map. The current NI specification and the older Quick Start differ on optional pins, so consult the exact hardware revision before wiring unused positions.

Pin	NI specifications, updated June 2025	Quick Start 376701A, December 2016
1	NC	NC
2	CAN_L	CAN_L
3	V-: CAN reference ground	COM
4	NC	NC
5	Optional shield	NC
6	Optional COM reference ground	COM
7	CAN_H	CAN_H
8	NC	NC
9	Optional V+ / Ext_Vbat assignment	NC

Table 42: NI USB-8502 pin assignments from two NI publications. [N2,N3]

Note - For normal USB-8502 CAN wiring, use CAN_H (7), CAN_L (2) and the CAN-side reference (3). USB powers the interface. Do not apply supply voltage to pin 9 based on a generic CAN cable convention; the older USB-8502 Quick Start marks it NC. Confirm the matching revision and harness before using optional assignments.

Termination and bench wiring

Provide a 120-ohm resistor at each physical end of a high-speed CAN bus. The NI checkbox controls only this NI port's resistor; it does not detect or configure other nodes. An unpowered, isolated bus with two end resistors normally measures about 60 ohms between CAN_H and CAN_L. Ground/reference wiring must respect the ECU and interface manuals.

The two-port model's separate COMBICON connector is labeled GND, T0 and T1 for clock/trigger synchronization. It is not the CAN connector, LIN, a power input for this workflow or the isolated CAN COM. Signal Explorer does not configure those synchronization signals. [N1,N2]

NI references

[N1] NI USB-8502 product profile

[N2] NI USB-8502 Quick Start 376701A

[N3] NI USB-8502 Specifications

[N4] NI Python binding installation

[N5] python-can NI-XNET backend

7.8 Ethernet connector reference

PC Ethernet: conventional 8P8C / RJ45 interface

Follow the jack/cable manufacturer's pin numbering; latch-side and contact-side views are different. The table is the conventional MDI signal assignment, not a promise about fixed TX/RX direction on auto-MDI-X hardware. Ethernet is not a D-Sub interface. [E3]

Pin	10/100BASE-T MDI	1000BASE-T pair
1	TX+	TP0+
2	TX-	TP0-
3	RX+	TP1+
4	Not a 10/100 data contact	TP2+
5	Not a 10/100 data contact	TP2-
6	RX-	TP1-
7	Not a 10/100 data contact	TP3+
8	Not a 10/100 data contact	TP3-

Table 43: Conventional copper Ethernet assignments. Gigabit uses all four bidirectional pairs; auto-MDI-X can exchange MDI roles. [E3]

Note - Unused-for-data does not mean safe for arbitrary power. PoE and proprietary ports can use these contacts. Never apply ValueCAN VBAT or LIN supply directly to an Ethernet data pair. PC NIC isolation, PoE capability and link speeds require the exact NIC manual.

ValueCAN4-2EL cable contacts

Signal	Dedicated DB26HD	Dedicated OBD-II / MULTI OBD-II
ETH TX+ / TX-	1 / 9	3 / 11
ETH RX+ / RX-	11 / 20	12 / 13
ETH ACTIVATE	25	8; listed for identification, not app-controlled
VBAT / ground	19 / 10	16 / 5

Table 44: Subset of the full ValueCAN connector maps in sections 7.5-7.6; verify the exact harness. [H11A,H11B,H11C]

Use the approved Intrepid Ethernet breakout for the actual DB26/OBD/MULTI cable. These contact numbers identify the vendor cable, not an instruction to wire a generic OBD connector or RJ45 lead. LIN and CAN pins remain as listed in their model-specific tables.

The ValueCAN4-2EL PHY is conventional 10/100 Ethernet. Automotive 100BASE-T1 requires an appropriate converter, such as the vendor's RAD-Moon accessory; a passive RJ45-to-single-pair cable is not a PHY converter. No native 100BASE-T1 PC adapter backend is supplied. [H2]

7.8a NI USB-8473 connector reference

The high-speed CAN port uses a 9-pin male D-Sub. Match the molded pin numbers on the adapter and use twisted pair for CAN_H and CAN_L.

Pin	Signal used for bench CAN
2	CAN_L
3	V- / CAN reference ground
7	CAN_H

Table 45: Core CAN wiring. Use the NI diagram for the other contacts.

For the remaining contacts, use the vendor pinout for your exact hardware revision. Terminate both physical ends of the bus with 120 ohms per end. The USB-8473s synchronization connector is separate from the CAN connector.

NI high-speed CAN pinout and wiring

7.9 Command line and configuration

Launch commands

```
cd "C:\Tools\Signal Explorer"
.\.venv\Scripts\python.exe -m signal_explorer
```

The installed console entry point is also **signal-explorer** when the virtual environment Scripts folder is on PATH. SignalExplorer.bat resolves its own folder and uses its local .venv interpreter; no PC-specific launcher path is needed.

Application arguments: none implemented

There is no application argument parser. Do not use invented options such as --adapter, --channel, --bitrate, --dbc, --log or --replay. Select those settings in the GUI. QApplication receives sys.argv and may process its own Qt options; those are not Signal Explorer connection features.

Configuration	Meaning
SIGNAL_EXPLORER_VECTOR_DLL	Absolute path to the installed matching XL DLL, normally vxlapi64.dll. Driver discovery exposes its folder before Vector import. Restart after changes.
Vector DLL button / QSettings	The GUI stores drivers/vector_dll in app settings. The environment variable takes priority.
SIGNAL_EXPLORER_PLIN_DLL	Absolute path to installed PLinApi.dll when it is not found through normal Windows library loading.
QT_QPA_PLATFORM=offscreen	Used for automated headless UI checks, not normal interactive operation. Do not set it in a normal launch session.

Table 46: Supported project environment hooks; vendor runtimes still must be installed.

Developer / verification commands

```
.\.venv\Scripts\python.exe -m pip check
.\.venv\Scripts\python.exe -m pytest tests -q
```

Automated checks use Virtual buses, simulation and mocked vendor APIs. They do not establish a new adapter's physical timing or electrical performance.

Where behavior is defined

ui/main_window.py: CAN controls; **ui/lin_panel.py**: LIN controls; **ui/*trace_model.py**: columns, counts and history; **bus/backends.py** and **bus/canfd.py**: rates/backend policy; **lin/vector.py**, **lin/intrepid.py**, **lin/peak.py**: native LIN; **ui/ethernet_panel.py** and **ethernet/**: Ethernet discovery, PC Npcap/Intrepid transports, simulation and raw-frame codec; **recording/**: MDF4 capture/replay. All paths are under src/signal_explorer.

Regenerate this guide with docs/build_users_guide.py using Python with reportlab and pypdf. The stored screenshots were captured with simulated traffic only. Recheck source and vendor revisions after software, hardware, firmware or cable changes.

7.10 Source register

Existing CAN/LIN sources were checked October 25, 2026; Ethernet additions were checked October 26, 2026. Match the serial, hardware revision and cable/piggy to the cited map. Test-voltage ratings are not interchangeable working-voltage ratings.

[H1] CANable product / getting-started pages

[H2] Intrepid ValueCAN4-2EL specifications

[H3] Intrepid ValueCAN4 family and model pinouts

[H4] Intrepid ValueCAN3 product / pinout

[H5] Vector-authored VN1600 family manual (vendor download)

[H6] Vector-authored Accessories Manual v5.4, pp. 42-43 and 57-59 (mirror)

[H7] Kvaser U100-C User's Guide, pp. 8, 11 and 14

[H9] PEAK PCAN-USB Pro FD User Manual, connector and technical sections

[H10] PEAK PLIN API reference

[H11] Intrepid 2EL / ValueCAN4 connector maps (including MULTI)

[H11A] Intrepid source diagram: 2EL DB26HD pin table

[H11B] Intrepid source diagram: dedicated 2EL OBD-II pin table

[H11C] Intrepid source diagram: MULTI DB9 / DB25 / OBD-II tables

[H11D] Intrepid source diagram: ValueCAN4-4 DB26HD

[H11E] Intrepid source diagram: ValueCAN4-4 OBD-II

[F1] Kvaser CAN error handling

[E1] Npcap installation and user guide

[E2] Npcap API reference: capture and packet injection

[E3] Oracle Ethernet connector pinouts, sections E.3-E.4

[H8] User-supplied Kvaser Leaf Light R v2 User's Guide, revision 3.10, printed October 7, 2026, pp. 9-11; *kvaser_leaf_light_r_v2_usersguide_3_10_261007.pdf*. This is also the formatting reference. Its specification table gives galvanic isolation without a numerical voltage rating.

Vector's official CDN rejected the local download during preparation. Vector-authored manual text was checked via an indexed manual mirror; for current revision details, use the official manual/Hardware Manager before wiring. Conditional piggy tables are examples and do not identify the user's two uninspected LIN piggies.

7.11 Revision history and validation

Revision / date	Change
1.9 / November 13, 2026	NI-CAN USB-8473/8473s setup, bench checks and wiring. Driver instructions with vendor links in the About dialog.
1.8 / November 10, 2026	Message Tree and live filtering, resizable DBC preview, remembered layout, bit and endpoint labels, CAN signal overlays and unsaved recording prompt.
1.7 / November 5, 2026	Opt-in driver TX reports, CAN timestamp provenance, monotonic host estimates and bounded recorder reordering with correction diagnostics.
1.6 / November 5, 2026	Busy-bus CAN batching and cached DBC/tree lookup; retained script history, drop counters, flush/capture filters, local TX timestamps and notification-driven waits.
1.0 / October 25, 2026	Initial CAN/FD/LIN guide, workflows, fault recovery, adapter pinouts and CLI.
1.1 / October 26, 2026	Ethernet simulation, PC Npcap and ValueCAN4-2EL: installation, discovery, GUI map, sending/filtering, MF4/replay, diagnostics, connectors and validation scope.
1.2 / October 29, 2026	Python editor/runner and CAN API; continuous 321/221 demo; File Viewer and read-only timed MF4 replay; SE icon/splash; Intrepid clock alignment and replay discontinuity/timeout fixes. Companion Scripting User's Guide and nine source examples added.
1.3 / October 30, 2026	NI-XNET USB-8502 Classic/FD integration, interface discovery and internal termination control; driver setup, NI MAX checks, electrical profile, pinouts and mock-test acceptance scope. NI-CAN excluded.
1.5 / November 2, 2026	Selective and remembered CAN channels; CAN Channel trace labels and default latest-value view; live raw/DBC multi-message TX, waveforms and JSON lists; channel-aware Python and MF4 recording/replay. VN1630A Classic loopback verified.
1.4 / October 31, 2026	Alphabetical per-protocol adapter lists, Virtual first-run defaults and remembered selections. DBC editor reference, bit layout, offline decoding, editing workflow, validation and save/reload behavior.

Table 47: Document revision, independent of the application version.

VN1630A serial 604069: one Classic CAN frame at 500 kbit/s, ID 5A1 and bytes A5 5A 01 02 03 04 05 06, was received on CAN2 after CAN1 TX. Physical FD and multi-message qualification remain pending; FIRE 3 inventory/routing uses mocks.

Signal Explorer is the application name used here. Vendor names identify supported interfaces and source documents; this guide does not imply vendor endorsement, product certification or independent ASAM conformance certification.

Validation for this edition

230 automated tests cover adapter preference restoration, DBC editing/validation/save isolation, the existing protocols, NI-XNET discovery/native session conversion and MF4 RX/TX, editor controls, pause/stop, replay direction/timing, log loading, timestamp alignment and the documented script examples. Five examples run against built-in virtual traffic; four MCU templates run against a private mock firmware peer after enabling their contract flag in the test harness. The default MCU templates are also checked to reject transmission before configuration. Native electrical qualification remains outside these tests.

docs/can-channels.md and docs/transmit-list.md describe the November 2 channel/TX behavior. docs/scripting.md, docs/file-viewer.md and examples/scripts/mcu/ describe testing and replay. Guide generators and capture scripts are in docs/. Dates above distinguish software/document updates from earlier hardware-source checks.